Agent-Based Modeling & Social Theory

HNRS-251-A - Honors Computer Science + Social Sciences

Eric Araújo, and **Jonathan Hill**

Keywords agent-based modeling, social sciences, computational methods

1. Course Policies

1.a. Grading Rubric

Welcome to the grading rubric for the Agent-Based Modeling & Social Theory course! Here's how your grade will be determined, with a friendly visual to help you see the big picture at a glance.

1.a.i. *Grading Breakdown:*

Component	% of Grade	Details
Structured Reading Groups	24%	Prep Sheets: 8 graded at 3% each. Lowest grade dropped.
Labs	20%	5–6 short lab memos (1–2 pages). Each worth 3–4%.
Reflection Essays	16%	Four essays, 1,000+ words each, 4% each.
Final Project	30%	Proposal, schema, prototype, review, presentation, report.
Participation & Contribution	10%	Seminar presence, peer feedback, teamwork evaluation.

> **Note**
>
> Labs and Reflection Essays are due the week after they are assigned before the start of the next class.

1.a.ii. *Details:*

*1. Structured Reading Groups — **24%**:*

- **Prep Sheets:** 8 graded at 3% each (lowest grade dropped, or one "freeloader").

*2. Labs — **20%**:*

- 5–6 short lab memos (1–2 pages), each worth 3–4%.
- Some labs may be interactive or use tools developed for the course.

*3. Reflection Essays — **16%**:*

Published Sep 22, 2025

Four essays, minimum 1,000 words each, worth 4% each.

- **Essay #1 (Week 3):** Durkheim & Weber (objectivity, emergence, knowledge).
- **Essay #2 (Week 6/7):** Collective Action / Cooperation (ABM for social processes).
- **Essay #3 (Week 10/11):** Polarization & Renewal (ABM, reconciliation, common good).
- **Essay #4 (Week 14):** Ethics (O'Neil, modeling and justice).

*4. Final Project — **30%**:*

- **Proposal:** 1%
- **Schema & Pseudocode:** 4%
- **Prototype Demo:** 5%
- **Design Review:** 5%
- **Final Presentation:** 10%
- **Final Report:** 5%

*5. Participation & Contribution — **10%**:*

- Seminar presence, peer feedback, peer evaluation for project teamwork.

> **Tip**
>
> Stay engaged, ask questions, and collaborate with your peers. We're here to help you succeed!

If you have any questions about grading, please reach out to your instructors.

1.b. Classroom Policy

> **Ask Questions & Participate!**
>
> Your voice matters in this classroom. Please ask questions, share your ideas, and participate actively.

1.b.i. ♿ *Disabilities & Accommodations:*

- Calvin University is committed to access for all students.
- If you have a documented disability, contact Student Success (Hiemenga Hall 227) to discuss accommodations.
- For pregnancy-related accommodations, contact the Title IX Coordinator (Spoelhof University Center 364).
- If you have an accommodation memo, please talk to me in the first two weeks of class.

1.b.ii. 🌍 *Diversity and Inclusion:*

As your instructor, I am committed to creating a welcoming learning environment for ALL students. It is my desire that students from all backgrounds and perspectives be served well in this course. I believe the diversity of experience and perspective students bring to this class enriches us all. This means that we all (myself included) need to practice humility, grace, and a posture of learning from others in the classroom. If you notice someone who is behaving in a way that consistently violates this spirit of inclusion and respect, please let me know.

1.b.iii. *Concerns Statement:*

If you or someone you know in my class is hurt by something we say or do, please let us know so that we can learn from our mistakes and work towards a resolution with you. We realize that confronting a professor can be intimidating and awkward, so if you do not feel you can approach any of us directly, feel free to do so through another student or another professor or staff member. My hope is that any cause for concern in our course could be dealt with within the confines of our class. However, if you experience something in our course or another course at Calvin that is particularly egregious and you do not feel safe engaging with me/the professor directly or indirectly, you can submit a complaint using the "Comment on Faculty" form online: https://calvin.edu/go/comments-on-faculty, which will be sent directly to the appropriate Dean. You may also use this form to express appreciation for faculty who have gone above and beyond to serve you well in some way.

1.b.iv. 🩺 *Illness, Absences, and Make-Up Work:*

If you fall ill (physically or mentally) for an extended period of time, **and** you send me documentation from Student Life, Student Health Center, or the Center for Health and Wellness, then—and only then—I will consider allowing you to make up late assignments.

> **Warning**
>
> If you begin to skip class repeatedly due to mental health problems, go to the Center for Health and Wellness! **Don't delay!**

1.b.v. ⏳ *Incompletes & Late Work:*

An Incomplete (I) grade will be granted only in unusual circumstances, and only if those circumstances have been verified by the Student Life Office. Procrastination does not qualify as an unusual circumstance.

> **Attention**
>
> **No work** will be accepted after the last day of classes besides the final project report.

1.c. Honesty

In this course, you will engage in hands-on modeling, coding, and critical reflection about social systems. Academic honesty is essential—not just for your own learning, but for the integrity of our shared exploration. The guidelines below clarify what is expected regarding collaboration, originality, and attribution in this interdisciplinary setting.

Collaboration & Independent Work:

- Some assignments (such as labs or group projects) will involve collaboration, while others (such as essays or individual coding tasks) must be completed independently. The instructions for each assignment will specify what kind of collaboration, if any, is permitted.
- When collaboration is allowed, you must still write up your own understanding and give credit to your collaborators.
- For individual work, all writing, code, and analysis must be your own.

1.c.i. *Plagiarism and Attribution in Social Modeling:*

- In this course, plagiarism includes copying code, text, analysis, or model structure from any source (including classmates, online repositories, or generative AI) without proper attribution.
- If you use ideas, code, or text from any source—including books, articles, websites, or AI tools—you must clearly cite the source in your submission.
- When in doubt, cite your source! Proper attribution is a sign of academic integrity and intellectual honesty.

Consider these rules of thumb:

- If you found it efficient to use copy/paste or use a generative language model to create more than one or two lines of your application, you must document the original source of the code.
- If the moment you figure out how to do something occurs while you are looking at a website or at the output of a generative language model, you should document that website.

Note that these rules of thumb apply to the code supplied in this course's materials as well.

1.c.ii. *Examples:*

Acceptable:

- Discussing general modeling strategies or social theory concepts with classmates.
- Citing and building on published models, as long as you clearly reference the source and explain your own contributions.

- Using generative AI or online resources for inspiration, provided you document exactly what was generated or copied and how you used it.

Unacceptable:

- Submitting code, text, or analysis from another student or online source as your own, without attribution.
- Copying large sections of code or text from generative AI or websites and failing to cite the source.
- Allowing someone else to submit your work as their own.

Warning

Our graders will be using a tool called the **Measure of Software Similarity (MOSS)** to check all submissions against all other submissions. If two submissions are found to be significantly similar, we will assume cheating has happened. Note that this software is pretty smart. It knows what computer language it is inspecting and can determine if two submissions are the same except for differences in variable names, e.g.

Note that if you and someone else both independently ask ChatGPT or Copilot (or some other LLM) to write code for you, and you both submit it, MOSS will detect it as identical and we will have to assume you cheated.

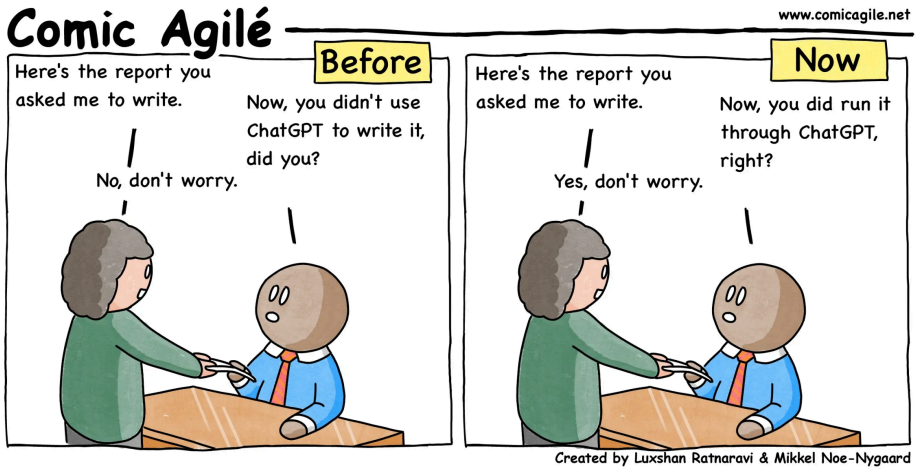


Figure 1: ChatGPT in the wild (comicagile.net)

1.c.iii. *Detection & Consequences:*

- All submissions may be checked for similarity using tools like MOSS or other plagiarism detection software.
- If significant overlap is found between your work and another source (including classmates, online code, or AI-generated content), it will be treated as academic dishonesty.

- Academic dishonesty may result in a failing grade for the assignment or the course, and will be reported according to Calvin's Academic Honesty policy.
- If you are ever unsure about what is allowed, ask your instructor before submitting your work.

2. Course Organization

This course is organized in **modules**. Each module typically lasts 2 weeks and includes readings, lectures, labs, and assignments. The course culminates in a final project where you will apply what you've learned to a social phenomenon of your choice using agent-based modeling.

2.a. Weekly classes

Day	Session	Time
Tuesday	Session A	2:10–2:55 pm
Tuesday	Break	2:55–3:05 pm
Tuesday	Session B	3:05–3:50 pm
Thursday	Session A	2:10–2:55 pm
Thursday	Break	2:55–3:05 pm
Thursday	Session B	3:05–3:50 pm

2.b. Important Dates

Date(s)	Event/Note
Tue Sept 2	Semester begins
Tue–Wed Oct 21–22	Advising Days (no class Tue Oct 21)
Wed–Fri Nov 26–28	Thanksgiving Break (no class Thu Nov 27)
Thu Dec 11	Last day of class
Dec 13–18	Final Project Reports

2.c. Modules organization

Module	Title	Weeks Covered	Main Topics/Focus
Module 1	Introduction	Weeks 1–2	Intro to ABM, Course orientation, Social sciences, Models & methods, NetLogo basics
Module 2	Segregation	Weeks 3–4	Segregation in sociology, Schelling model, Inequality, Extending models
Module 3	Contagion	Weeks 5–6	Collective behavior, Thresholds, Social movements, Protest/ rebellion, Project pitches

Module 4	Cooperation	Weeks 7–8	Cooperation, Commons, IPD, Project scope, Proposal workshop
Module 5	Polarization	Weeks 9–11 (and parts of 12-15)	Diffusion, Polarization theory, Labs, Debates, Project build, Presentations

---

2.d. Visual Overview

Week	Dates	📖 SRG Prep	🧪 Lab Memo	🎓 Project Milestone	🗂️ Other/ Notes
1	Sep 2 & 4				Course intro
2	Sep 9 & 11	✅ #1	✅ #1		
3	Sep 16 & 18	✅ #2	✅ #2		
4	Sep 23 & 25	✅ #3	✅ #3		
5	Sep 30 & Oct 2	✅ #4	✅ #4		
6	Oct 7 & 9	✅ #5			
7	Oct 14 & 16	✅ #6	✅ #5		
8	Oct 21 & 23			✅ Proposal	
9	Oct 28 & 30	✅ #7		✅ Schema & Pseudocode	
10	Nov 4 & 6	✅ #8	✅ #6		
11	Nov 11 & 13	✅ #9	✅ #7		
12	Nov 18 & 20			✅ Alpha	
13	Nov 25			✅ Beta Demo	
14	Dec 2 & 4	✅ #10			Poster Draft
15	Dec 9 & 11			✅ Final Pres, Report	

Legend:

- 📖 SRG Prep = Structured Reading Group Prep Sheet
- 🧪 Lab Memo = Lab Memo
- 🎓 Project Milestone = Project Proposal, Schema, Alpha, Beta, Design Review, Final Presentation/Report

- 🗂️ Other/Notes = Poster Draft, Course intro, etc.

For the grading planning, visit **Course Policies** arrow.r **Grading Rubric**.

3. Course Modules

3.a. Module 1: Introduction

Welcome to the **Introduction to Agent-Based Modeling & Social Theory** module! This foundational module introduces the intersection of computer science and social science, exploring how computational methods can help us understand human behavior and social phenomena.

This module provides the theoretical and practical foundations for the course. We'll explore what models are, why they're useful in social science research, and how computational approaches can reveal insights about complex social systems. We will also introduce agent-based modeling as a key method for simulating social phenomena.

Module Duration: 2 weeks

3.a.i. *Student Learning Objectives (SLOs):*

By the end of this module, students will be able to accomplish the following SLOs:

Core SLOs

- Introduce students to observational research methods for gathering empirical evidence and to theories that provide analytical frameworks for such evidence.
- Provide students with a sense of the nature and limits of scientific knowledge and the kinds of ethical questions that surround scientific research and its dissemination.
- Study how ideas from mathematical sciences have reflected and shaped other ways of thinking and knowing.
- Define what constitutes a model in social science research

Conceptual

- Explain the relationship between social computing and traditional social science methods
- Reflect on the idea of *emergence* introduced by Durkheim and how it sets the stage for ABM.
- Identify strengths and limitations of computational modeling approaches
- Understand the role of abstraction in model building
- Comprehend the relevance of using Agent-based models for simulating social phenomena

Technical

- Get to know Netlogo's interface and visualize some classic models

- Install Netlogo and run your first simulation from the Library of models

Critical Thinking

- Evaluate when computational modeling is appropriate for social questions
- Assess the validity and reliability of model assumptions
- Connect abstract models to real-world social phenomena
- Critique modeling choices and their implications

Communication

- Articulate the purpose and value of social modeling
- Initiate discussions on interpreting model results to both technical and non-technical audiences
- Discuss ethical considerations in social modeling research
- Engage with interdisciplinary perspectives on social phenomena

---

3.a.ii. 📋 *Weekly Breakdown:*

Lecture 1

Week 1: Tuesday, September 2

Heckman Library 406C

Session A:

- **Summary:** What is ABM? Live demo in NetLogo (Starling's murmuration, Wolf-Sheep Predation, and fire model).
- **Slides:** Introduction to Models: What is ABM?

Session B:

- **Summary:** Course orientation. Syllabus overview, grading, expectations.

Introduction to project & SRGs.

- **Slides:** Course Orientation

Lecture 2

Week 1: Thursday, September 4

Heckman Library 406C

Session A:

- **Summary:** Emergence of the social sciences; classical theories; fundamental debates.

Sociology distinct from psychology: focus on emergence and social facts (Durkheim).

- **Slides:** Emergence of the social sciences

Session B:

- **Summary:** Models and methods: Where does ABM fit? Connect emergence to ABM: micro arrow.r macro link as ABM's strength.

Discussion: "What kinds of social problems make sense only at the level of emergence?"

- **Slides:** ABM Fundamentals

Lecture 3

Week 2: Tuesday, September 9

Heckman Library 406C

Session A (SRG):

- **Summary:** Discussion of readings (Durkheim & Weber).

SRG goal: Contrast Durkheim's irreducible social facts with Weber's abstraction/ ideal types.

Session B:

- **Summary:** Workshop — What makes a good model?

Groups evaluate "bad" and "good" toy models.
Criteria: simplicity, clarity, generativity, transparency, theory link.

- **Slides:** Models and Methods

Lecture 4

Week 2: Thursday, September 11

Heckman Library 406C

Session A (Lab): NetLogo basics—interface, parameters, behaviors.

Session B (Lab): Micro-exercise: Adjust one parameter in a simple model, note results.

3.a.iii. 📝 *Assignments & Due Dates (Weeks 1–2):*

Important ❗ ❗ ❗

No homework due in Week 1. The first major assignments are due at the end of Week 2.

Lab Memo 1

Due: 9/16 before class | **Points:** 20 points

Prompt (1-2 pages):

1. Pick one of the toy models shown in our classes over these two weeks (except the Fire model), and perform a single parameter adjustment.
    1. Setup: Identify which starter model you used (e.g., Wolf-Sheep, Traffic, Fire).
    2. Parameters & Code: Adjust one parameter; note what you changed. If you tried editing code, briefly explain what.
    3. Results: Describe what happened compared to the default. Include screenshot(s).
    4. Interpretation: What does this show about how simple rules create different outcomes?
2. Write your Lab Memo. You can download the template in here.
3. Submit your Lab Memo in PDF format through Moodle.

Reflection Essay 1

Due: 9/16 before class | **Points:** 30 points

Prompt (≥1000 words):

- How do Durkheim and Weber differ in their approaches to building knowledge?
- Where do you see common ground?
- How might ABMs fit into ongoing discussions about subjectivity, objectivity, and building valid social knowledge?

3.a.iv. 📚 *Readings and Extra Materials:*

🔒 **Required Readings**

1. **Halls, W. D., & Lukes, S.** (1982). Durkheim:The Rules of Sociological Method and Selected Texts on Sociology and Its Method. Excerpt.
    - 📖 PDF Download

2. **Weber, M.** (1949). “ Objectivity” in social science and social policy. The methodology of the social sciences, 49-112. Excerpt.
    - 📖 PDF Download

🔒 **Recommended Readings**

1. **Smaldino, P.** (2023). Modeling social behavior: Mathematical and agent-based models of social dynamics and cultural evolution. Chapter 1.
2. Netlogo Programming Guide.

📽️ **Inspirational Videos**

- 🎥 The Power of Models (4 min)
- 🎥 Top 3 aspects people get wrong about Agent Based Modeling (9 min)
- 🎥 When is a system complex? (3 min)
- 🎥 Emergence – How Stupid Things Become Smart Together (7 min)

Real-World Applications:

3.b. Module 2: Segregation

> **Warning**
>
> This page is under construction. It is just a draft that will receive a lot of changes yet.

Welcome to the Segregation Models module! This section explores computational models of residential segregation, building on Thomas Schelling's groundbreaking work Schelling (1971a) on how individual preferences can lead to collective patterns of segregation. We will generalize the residential model and see ways to apply it to other contexts.

Segregation models help us understand how micro-level individual choices can lead to macro-level social phenomena Schelling (1978). Through agent-based modeling, we'll explore how even mild preferences for similarity can result in highly segregated neighborhoods, providing insights into urban dynamics and social processes.

Module Duration: 2 weeks

3.b.i. *Student Learning Objectives (SLOs):*

By the end of this module, students will be able to:

Core SLOs

- Increase students' knowledge of social systems and of human behavior within such systems
- Apply algorithmic, statistical, and/or mathematical methods to solve problems, broadly defined to find the answers to questions in various domains (as appropriate).
- Represent, interpret, and process information in graphical, numeric, and/or symbolic forms.

Conceptual SLOs

- Explain the difference between individual preferences and collective outcomes
- Analyze how threshold models work in social systems
- Evaluate the relationship between micro-motives and macro-behavior

Technical SLOs

- Navigate the NetLogo interface and basic programming concepts
- Create simple agent-based models with basic behaviors
- Run simulations and collect data from model outputs
- Interpret basic visualization and data outputs

- Implement Schelling's segregation model in NetLogo
- Modify agent rules and parameters to test different scenarios
- Collect and analyze data from agent-based simulations

Critical Thinking

- Assess the implications of segregation models for real-world policy
- Compare model predictions with empirical data on residential patterns
- Critique the assumptions and limitations of segregation models

Communication

- Present findings from simulation experiments clearly
- Discuss ethical implications of segregation research
- Connect model insights to contemporary social issues

3.b.ii. 📋 *Weekly Breakdown:*

Lecture 5

Week 3: Tuesday, September 16

Heckman Library 406C

Session A: *History of segregation in social research.*

- **Summary:**
  - Traditions: Du Bois' *The Philadelphia Negro*, Chicago School, Massey & Denton's structural view.
  - Concepts: de jure vs. de facto segregation; neighborhood effects; systemic racism.
- **Slides:** Segregation

Session B: Discussion of readings.

- **Summary:**
  - Massey & Denton, *American Apartheid* (1993), Ch. 1.
  - Banaji, Fiske & Massey, "Systemic Racism" (*Cognitive Research*, 2021).
  - Deliverable: SRG prep sheet.

Lecture 6

Week 3: Thursday, September 18

Heckman Library 406C

Session A (Lab): Schelling segregation model.

- **Summary:** Building Schelling's model
- **Slides:** Schelling Model

Session B:

- **Summary:**
  - Explore tolerance thresholds, group asymmetry, neighborhood sizes.
  - Class critique: What's realistic? What's missing (e.g., structural constraints)?
  - Deliverable: **Lab Memo #2**.
- **Code:** Netlogo Code developed in class

Lecture 7

Week 4: Tuesday, September 23

Heckman Library 406C

Session A (SRG):

- **Summary:** TBD
  - Bruch, Elizabeth E., and Robert D. Mare. "Neighborhood choice and neighborhood change." American Journal of sociology 112.3 (2006): 667-709.

Session B:

- **Summary:** TBD
- **Slides:** TBD

Lecture 8

Week 4: Thursday, September 25

Heckman Library 406C

Session A (Lab): TBD

Session B (Lab): TBD

3.b.iii. 📝 *Assignments & Due Dates (Weeks 3–4):*

Lab Memo 2

Due: 9/23 before class | **Points:** 20 points

Prompt (1-2 pages):

1. Take the code for the Schelling model implemented in class in the link above (Lecture 6). Your task is to modify the model in some way and analyze the results. You can choose one of the following options:
    - **Add a reporter:** e.g., track the number of moves, average satisfaction, or segregation index over time.
    - **Change the neighborhood definition:** e.g., use a larger or smaller neighborhood size.
    - **Introduce heterogeneity:** e.g., allow agents to have different tolerance levels or preferences.
    - **Add mobility constraints:** e.g., limit how far agents can move in a single step.
2. Write your Lab Memo. You can download the template in here.
3. Make sure you add the codes you've changed, as well as interface modifications.
4. Submit your Lab Memo in PDF format through Moodle.

3.b.iv. 📚 *Readings and Extra Materials:*

🔒 **Required Readings**

1. **Bruch, Elizabeth E., and Robert D. Mare.** "Neighborhood choice and neighborhood change." American Journal of sociology 112.3 (2006): 667-709.
    - 📖 PDF Download

🔒 **Recommended Readings**

1. **Bruch, Elizabeth E.** "How population structure shapes neighborhood segregation." American Journal of Sociology 119.5 (2014): 1221-1278.
    - 📖 PDF Download
2. **Schelling, T. C. (1971).** *Dynamic models of segregation.* Journal of Mathematical Sociology, 1(2), 143-186.
    - 📖 Schelling (1971b)

📽️ **Inspirational Videos**

- 📹 American Segregation, mapped at day and night (7 min)
- 📹 The Power of Models (4 min)
- 📹 Top 3 aspects people get wrong about Agent Based Modeling (9 min)

- 🎥 When is a system complex? (3 min)
- 🎥 Emergence – How Stupid Things Become Smart Together (7 min)

3.c. Module 3: Contagion

> **Warning**
>
> This page is under construction. It is just a draft that will receive a lot of changes yet.

Welcome to the Contagion Models module! This module explores how ideas, behaviors, diseases, and innovations spread through social networks and populations. We'll examine both biological and social contagion processes using computational modeling approaches.

3.c.i. *Overview:*

Contagion models help us understand how things spread - from infectious diseases to social movements, from rumors to technological innovations. Through agent-based modeling, we'll explore different mechanisms of transmission, the role of network structure, and intervention strategies for controlling or promoting different types of contagion.

> **Note**
>
> **Module Duration:** 2 weeks | *Estimated time: 8-12 hours*

3.c.ii. *Student Learning Objectives (SLOs):*

By the end of this module, students will be able to:

Core

- Construct data-driven, mathematical, statistical, and/or software models, analyzing their results to answer questions, solve problems, support arguments, draw conclusions, make predictions, and/or identify possible causal relationships.
- Identify and use tools appropriate for solving a given problem, such as algebra, calculus, and other mathematical tools; spreadsheets, databases, and data-analysis software; domain-specific software; and/or writing one's own software.

Conceptual

- Distinguish between different types of contagion (biological, social, behavioral)
- Explain the role of network structure in contagion processes
- Analyze the dynamics of epidemic curves and tipping points
- Understand concepts like basic reproduction number (R_0) and herd immunity

Technical Skills

- Implement SIR and SEIR epidemiological models in NetLogo
- Model contagion on different network topologies
- Analyze the effects of intervention strategies on spread dynamics
- Visualize and interpret contagion simulation results

Critical Thinking

- Evaluate the effectiveness of public health interventions
- Assess the parallels and differences between biological and social contagion
- Critique assumptions in contagion models and their real-world applicability
- Analyze the ethical implications of contagion research and policy

Communication

- Present epidemiological concepts to diverse audiences
- Discuss the role of modeling in public health decision-making
- Communicate uncertainty and risk in contagion scenarios
- Engage with contemporary debates about disease control and social influence

---

3.c.iii. 📚 *Slides and Readings:*

Required Readings:

3.c.iv. 📝 *Homework:*

3.c.v. 🌟 *Extra Materials:*

Historical Context:

Real-World Applications:

3.c.vi. 🗓️ *Weekly Schedule:*

3.c.vii. 📞 *Getting Help:*

3.d. Module 4: Cooperation

> **Warning**
>
> This page is under construction. It is just a draft that will receive a lot of changes yet.

Welcome to the Cooperation Models module! This module explores one of the most fundamental questions in social science: how and why do humans cooperate? We'll use game theory and agent-based modeling to understand the conditions that promote or hinder cooperation in social systems.

3.d.i. *Overview:*

Cooperation models help us understand how individuals can work together for mutual benefit, even when short-term self-interest might suggest otherwise. Through computational modeling, we'll explore classic cooperation dilemmas, evolutionary strategies, and the role of reputation, punishment, and reward systems in maintaining cooperative behavior.

> **Note**
>
> **Module Duration:** 2 weeks | *Estimated time: 8-12 hours*

3.d.ii. *Student Learning Objectives (SLOs):*

By the end of this module, students will be able to:

Core

- Develop students' understanding of biblically-guided norms of justice, equality, freedom, and stewardship.
- Increase students' knowledge of social systems and of human behavior within such systems (revisited in the context of cooperation and dilemmas).
- Apply algorithmic, statistical, and/or mathematical methods to solve problems (as applied to cooperation and social dilemmas).

Conceptual

- Explain the fundamental cooperation dilemmas (Prisoner's Dilemma, Public Goods, etc.)
- Understand evolutionary approaches to cooperation and reciprocity
- Analyze the role of institutions, norms, and sanctions in promoting cooperation
- Evaluate different mechanisms for solving collective action problems

Technical Skills

- Implement game theory models in NetLogo
- Model evolutionary strategies and fitness landscapes
- Simulate reputation systems and social learning mechanisms
- Analyze equilibrium outcomes and stability conditions

Critical Thinking

- Assess the conditions under which cooperation emerges and persists
- Evaluate the effectiveness of different institutional designs
- Critique the assumptions of rational choice and evolutionary models
- Connect cooperation theory to real-world social and political challenges

Communication

- Present game theory concepts to diverse audiences
- Discuss the implications of cooperation research for policy design
- Articulate the tension between individual and collective interests
- Engage with debates about human nature and social institutions

---

3.d.iii. 📚 *Slides and Readings:*

Required Readings:

3.d.iv. 📝 *Homework:*

3.d.v. 🌟 *Extra Materials:*

Historical Context:

Real-World Applications:

3.d.vi. 🗓️ *Weekly Schedule:*

3.d.vii. 📞 *Getting Help:*

3.e. Module 5: Polarization

> **Warning**
>
> This page is under construction. It is just a draft that will receive a lot of changes yet.

Welcome to the Polarization Models module! This module explores how groups form opposing viewpoints, how moderate positions erode, and how social identities become divided. We'll examine the mechanisms that drive political, social, and ideological polarization using computational modeling approaches.

3.e.i. *Overview:*

Polarization models help us understand one of the most pressing challenges of contemporary society: how and why groups become increasingly divided in their beliefs, values, and behaviors. Through agent-based modeling, we'll explore opinion dynamics, the role of social networks, media influence, and institutional factors that contribute to societal polarization and potential interventions.

> **Note**
>
> **Module Duration:** 2 weeks | *Estimated time: 8-12 hours*

3.e.ii. *Student Learning Objectives (SLOs):*

By the end of this module, students will be able to:

Core

- Articulate limiting assumptions or limitations to the conclusions that can be drawn from the use of these methods and identify appropriate and inappropriate uses of such methods, as informed by a Reformed, Christian perspective.
- Provide students with a sense of the nature and limits of scientific knowledge and the kinds of ethical questions that surround scientific research and its dissemination (revisited in the context of polarization, ethics, and justice).

Conceptual

- Define different types of polarization (ideological, affective, social sorting)
- Explain mechanisms that drive opinion polarization and group formation
- Analyze the role of social networks, media, and algorithms in polarization
- Understand the relationship between polarization and democratic governance

Technical Skills

- Implement opinion dynamics models (voter model, Hegselmann-Krause, etc.)
- Model the effects of network structure on opinion formation
- Simulate media influence and algorithmic filtering on belief systems
- Analyze polarization metrics and measurement approaches

Critical Thinking

- Evaluate competing explanations for political and social polarization
- Assess the effectiveness of interventions designed to reduce polarization
- Critique the assumptions and limitations of polarization models
- Connect polarization theory to contemporary political and social challenges

Communication

- Present complex polarization dynamics to diverse audiences
- Discuss the implications of polarization research for democratic society
- Articulate the tension between diversity and social cohesion
- Engage constructively with politically sensitive topics and research

---

3.e.iii. 📚 *Slides and Readings:*

Required Readings:

3.e.iv. 📝 *Homework:*

3.e.v. 🌟 *Extra Materials:*

Historical Context:

Real-World Applications:

3.e.vi. 🗓️ *Weekly Schedule:*

3.e.vii. 📞 *Getting Help:*

4. NetLogo Tutorial Series

> **Warning**
>
> Netlogo has recently released its 7.0 version. A lot of improvements were done in the interface, making it more user-friendly and accessible for beginners. That means that this tutorial might be outdated regarding some of the screen shots. The updates might take a few weeks.

> **Total time commitment:**
>
> Approximately 6 hours across 7 tutorials

Welcome to the NetLogo programming material designed specifically for this class. We assume students may not have any prior programming experience. And that's okay! We will walk through the basics of programming in NetLogo, and also learn in class what it feels like building code for modeling social behavior.

4.a. Course Overview

This course teaches agent-based modeling using NetLogo, a powerful yet accessible programming environment. You'll learn to create computational models that help answer questions about social phenomena.

This guide follows the same structure as the NetLogo documentation, but it is tailored to our course. It is not a replacement for the official documentation, but rather a complement to it. We own all the credit for this material to Ury Wilensky and the Center for Connected Learning and Computer-Based Modeling at Northwestern University Wilensky (1999). The official documentation is a great resource, and we encourage you to use it as well.

In this tutorial, we intend to provide the basic concepts and programming skills necessary to start programming in NetLogo. This tutorial is focused on programming. Come back to this part whenever you need to refresh your memory on the basic concepts of programming in NetLogo.

> **Note**
>
> **No programming background required!** This course is designed for students who have never programmed before. We start with basic concepts and build gradually.

4.a.i. *Learning objectives:*

By completing this tutorial series, you will be able to:

- Create agent-based models from scratch
- Understand how individual behaviors create collective patterns
- Use computational tools to explore social behavior questions
- Analyze and interpret model results

- Extend existing models for new research questions

4.a.ii. *Why NetLogo for social models?:*

- Visual programming environment reduces intimidation
- Designed specifically for modeling social phenomena
- Large community of non-CS users
- Extensive library of existing models to learn from

4.a.iii. *Learning Approach:*

This course uses a **learning-by-doing** approach:

- Start with exploration and experimentation
- Build understanding through hands-on activities
- Progress from simple concepts to complete models
- Focus on visual and conceptual understanding first

4.b. Tutorial Structure

Tutorial 0: Introduction & Interface (40 minutes)

- What is NetLogo and why use it?
- Exploring the NetLogo interface
- Running your first model

Tutorial 1: Basic Programming Concepts (50 minutes)

- Programming Without Fear (10 min)
- Core Concepts (25 min)
- Reading Code (15 min)

Tutorial 2: Working with Agents (60 minutes)

- Meet the Turtles (10 min)
- Creating and Controlling (20 min)
- Making Agents Unique (15 min)
- Agent Interactions (15 min)

Tutorial 3: Environment and Patches (40 minutes)

- Understanding Patches (10 min)
- Patch Properties and Visualization (15 min)
- Turtle-Patch Interactions (15 min)

Tutorial 4: Building Your First Complete Model (55 minutes)

- Planning Before Programming (15 min)
- Step-by-Step Model Building (30 min)
- Documentation and Sharing (10 min)

Tutorial 5: Data Collection and Analysis (45 minutes)

- Why Data Matters in Modeling (10 min)
- Using NetLogo's Built-in Analysis Tools (20 min)
- Exporting Data for External Analysis (15 min)

Tutorial 6: Advanced Topics and Troubleshooting (45 minutes)

- Common Problems and Solutions (20 min)
- Extending Models with New Features (15 min)
- Where to Go Next in Your Modeling Journey (10 min)

4.c. Getting Started

Begin with: Tutorial 0: NetLogo Introduction & Interface

Prerequisites: None! Just curiosity about social phenomena and willingness to experiment.

What you'll need:

- NetLogo software (free download from netlogo.org)
- Willingness to try things and make mistakes
- Notebook for observations and questions

4.d. Learning Path

> **Note**
>
> **Prerequisites:** Complete Tutorial 0: NetLogo Introduction & Interface before starting these programming tutorials.

Recommended progression:

1. Start with basic concepts and work through each tutorial sequentially
2. Complete all activities and exercises in each tutorial
3. Build the example models to reinforce learning
4. Use the troubleshooting guide when you encounter problems

Time commitment: Plan for approximately 5 hours total to complete all programming tutorials with activities.

4.e. Getting Help

If you get stuck:

- Review the troubleshooting section in Tutorial 6
- Check that you've completed prerequisites
- Try simplifying your code and building complexity gradually
- Ask questions during class or office hours

Ready to start exploring the world of agent-based modeling? Let's begin!

4.f. Introduction & Interface

Welcome to NetLogo! This tutorial will help you get comfortable with NetLogo in just 40 minutes.

We will start focusing on the interface and how to use existing models. Later, we will explore how to read and modify the code behind the models.

Commitment

Time Required: 40 minutes
Target: students learning agent-based modeling

Getting Started

1. **Download NetLogo** from https://ccl.northwestern.edu/netlogo/download.shtml
2. **Install and open** NetLogo
3. **Open Models Library:** File arrow.r Models Library

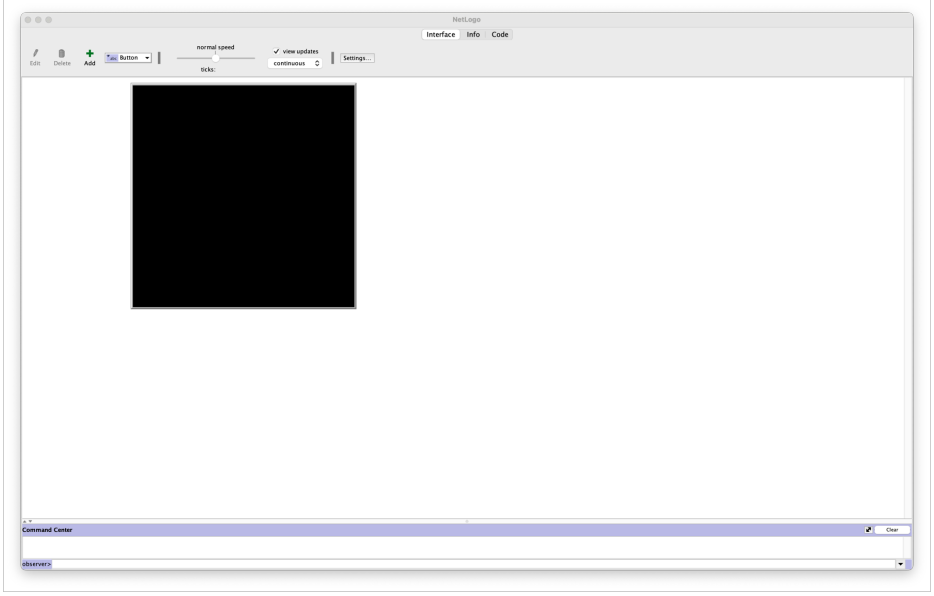


Figure 2: NetLogo interface when a new project is being started.

4.f.i. *What is NetLogo? (5 min):*

NetLogo is a **visual programming environment for agent-based modeling** - think of it as a virtual laboratory where you can:

- Create "agents" (people, animals, organizations) that follow simple rules
- Watch how individual behaviors create complex collective patterns
- Test "what if" scenarios by changing rules and seeing results

Used by Researchers Worldwide (Not Just Programmers!):

- **Economists** model market dynamics and trading behaviors
- **Sociologists** study how social norms spread through communities
- **Political scientists** explore voting patterns and coalition formation
- **Urban planners** analyze traffic flow and city development
- **Epidemiologists** track disease spread and intervention strategies

Examples of famous models built in NetLogo::

- Thomas Schelling's segregation model (Nobel Prize winner)
- Robert Axelrod's cooperation tournaments
- Models of financial markets and economic bubbles

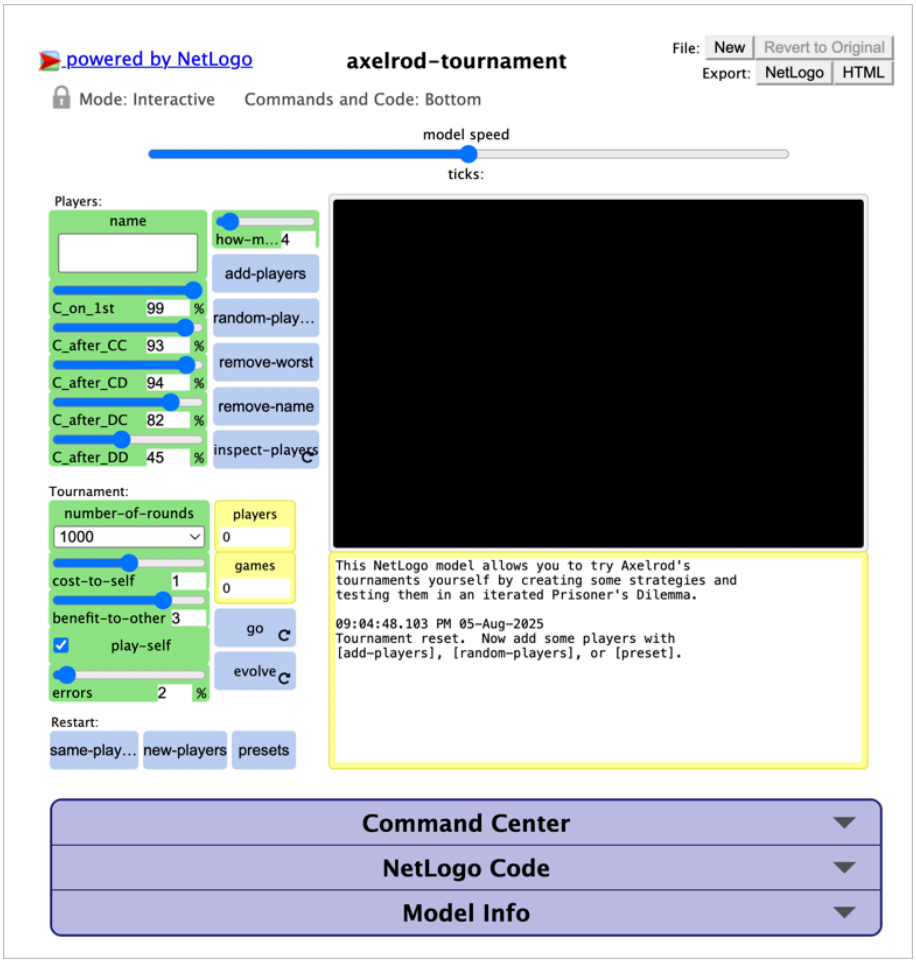


Figure 3: Robert Axelrod's cooperative tournament model implemented in NetLogo.

The last model you will have to download and run in your own NetLogo installation.

Spend some time exploring the models in the NetLogo Models Library. Click in the links above and read the Info tab for each model to learn more about what they do. Don't feel overwhelmed by the amount of information - just get a sense of the variety of models available.

4.f.ii. *Interface Tour (15 min):*

Let's explore the four main areas of NetLogo. We will use the Traffic Grid model as an example, but these concepts apply to all models.

Important

To open the **Traffic Grid model**, go to File arrow.r Models Library, then browse to "Social Science" arrow.r "Traffic Grid" and click "Open".

1. World View (where agents live and move):

- This is the "stage" where your social drama unfolds
- Agents appear as colored shapes that move and interact
- Think of it as looking down at a city from above

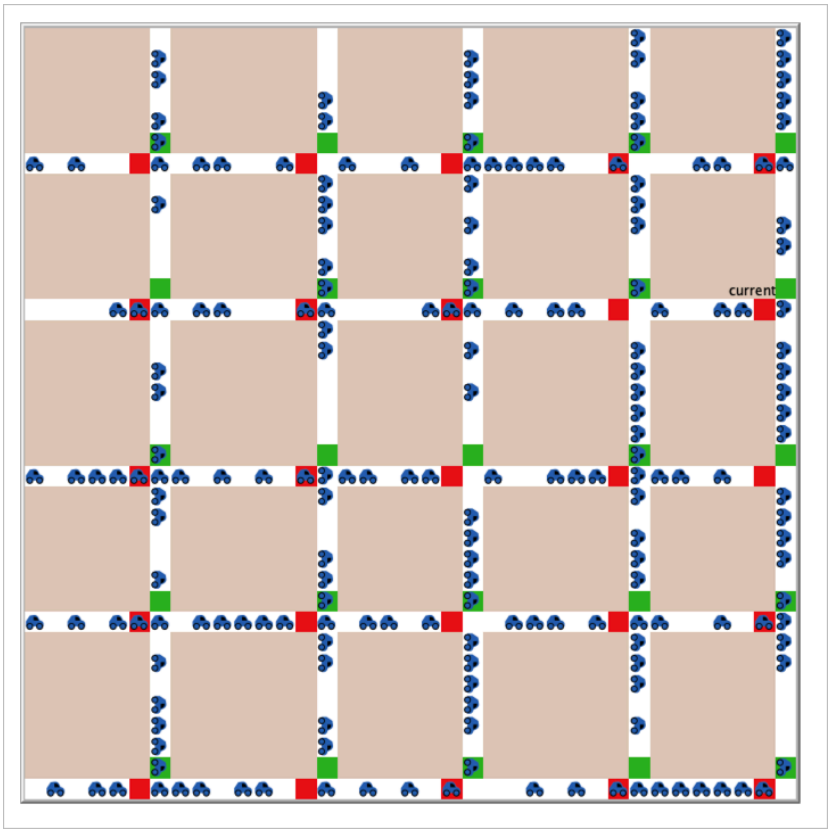


Figure 4: Traffic Grid model's world view.

2. Interface Tab (buttons, sliders, plots you can interact with):

The Figure below shows a typical model interface:

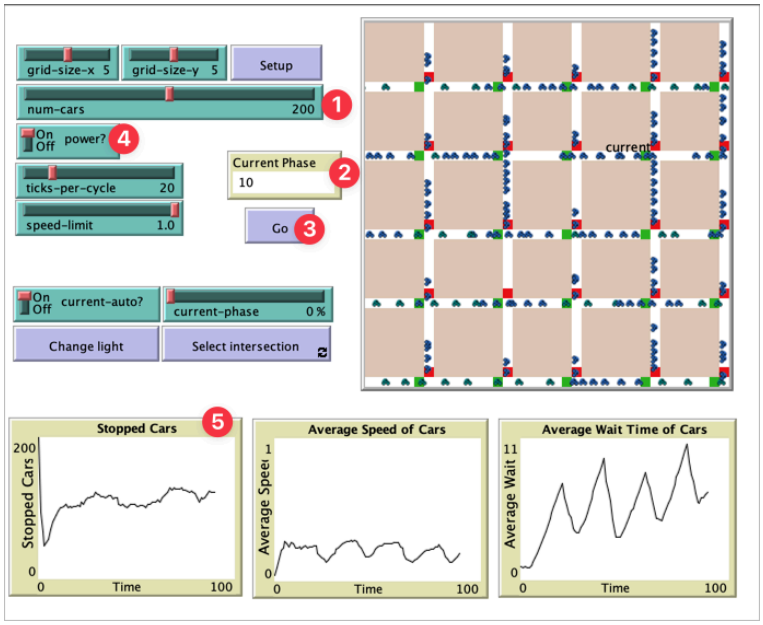


Figure 5: Example of a NetLogo model interface showing the visualization and controls.

- **Sliders (1):** Adjust parameters like "how many people?" or "how fast do they move?"
- **Monitors (2):** Display current values like "average wealth"
- **Buttons (3):** Start, stop, and control your model
- **Switches (4):** Toggle options on and off
- **Plots (5):** Graphs showing data over time

3. Code Tab (where the magic happens - but don't worry!):

- Contains the "rules" that tell agents what to do
- Uses English-like commands: `move-forward`, `turn-right`
- We'll start by reading code, not writing it

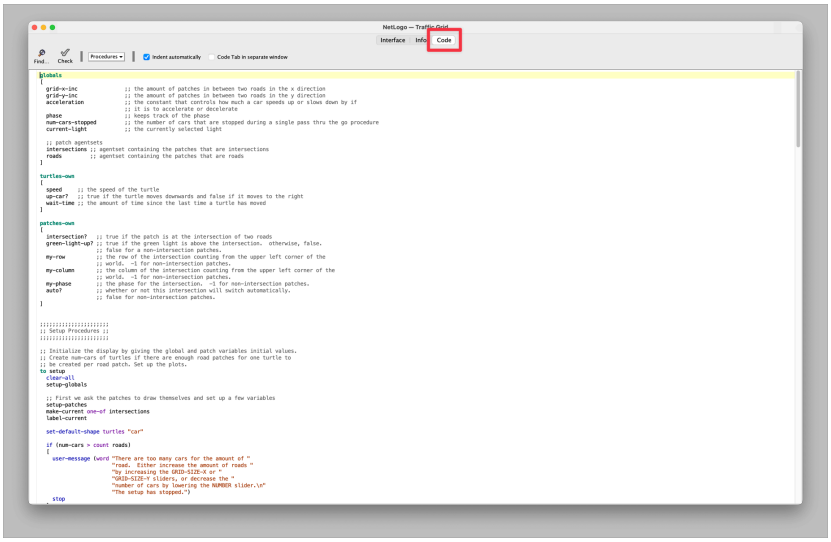


Figure 6: Example of a NetLogo Code tab interface showing the code for the Traffic Grid model.

4. Info Tab (documentation and explanations):

- Always read this first when exploring a model!
- Explains what the model does and what to look for
- Provides background and references

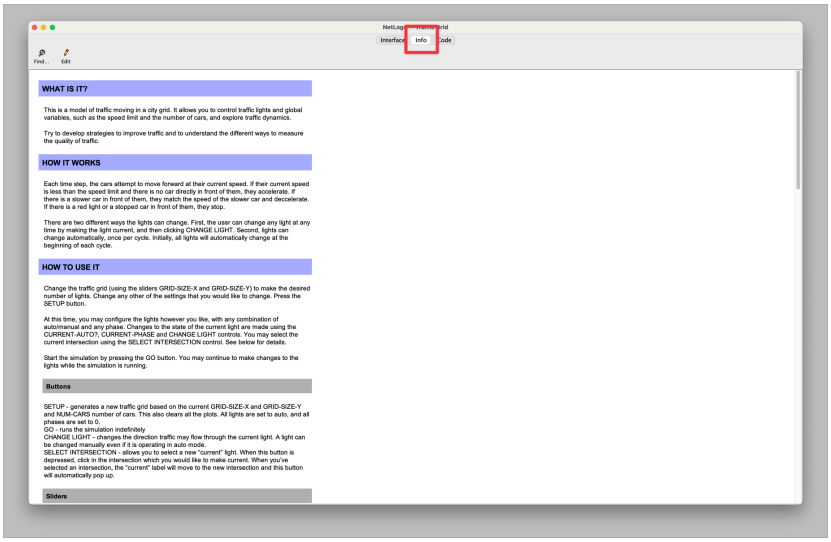


Figure 7: Example of a NetLogo Info tab interface showing model documentation and explanations.

4.f.iii. *Your First Model Experience (20 min):*

Let's explore classic models to see NetLogo in action.

Activity 1: Wolf Sheep Predation:

Step 1: Open the model

- File arrow.r Models Library
- Browse to "Biology" arrow.r "Wolf Sheep Predation"
- Click "Open"

Step 2: Explore the interface

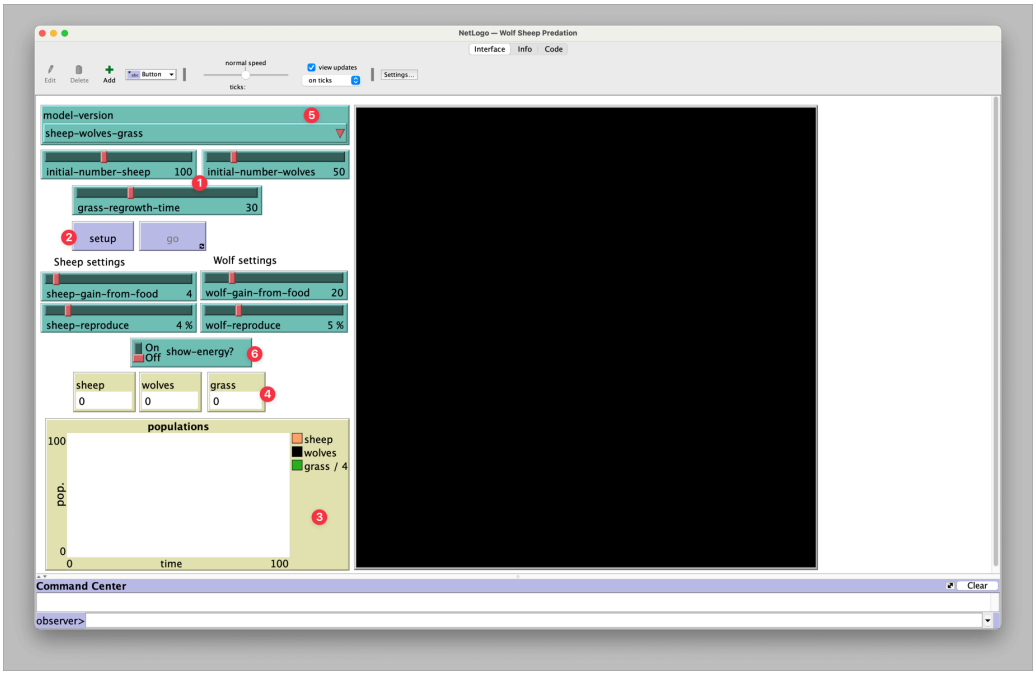


Figure 8: Wolf Sheep Predation model interface showing controls and visualization.

- **Sliders (1):** initial-number-sheep, initial-number-wolves, etc.
- **Buttons (2):** setup, go, go (forever)
- **Plots (3):** populations over time
- **Monitors (4):** current populations
- **Choosers (5):** select model version
- **Switches (6):** toggle features like "show energy"

Step 3: Run your first simulation

- Click **"setup"** (creates initial population)
- Click **"go"** several times (runs step-by-step)
- Or click **"go forever"** (runs continuously)

Step 4: Experiment with parameters

- Reset: Click “setup”
- Change “initial-number-wolves” to 250 (very high)
- Click “setup” then “go” - what happens?
- Try very few wolves (10) - how does this change things?

Activity 2: Exploration Exercise:

Your turn! Pick any model from the Models Library and explore for 10 minutes.

Suggested models for social scientists:

- **Segregation** (Social Science) - neighborhood preferences and segregation
- **Voting** (Social Science) - how voting behaviors spread
- **Cooperation** (Social Science) - prisoner’s dilemma strategies
- **Traffic Basic** (Social Science) - traffic jams from individual decisions

For each model:

1. **Read the Info tab** - what is this about?
2. **Run with default settings** - what happens?
3. **Change parameters** - how do outcomes change?
4. **Form a hypothesis** - “If I change X, then Y will happen”
5. **Test it** - were you right?

Share one surprising discovery: What model did you choose and what surprised you most?

4.f.iv. *What We've Accomplished:*

Learning Objectives Achieved:

By completing this tutorial, you can now:

✓ Navigate the NetLogo interface confidently
✓ Understand the difference between agents and environment
✓ Experience how changing parameters affects outcomes
✓ Feel comfortable exploring pre-built models

What's Next?:

In the next section, we'll peek "under the hood" to see how these models work. But don't worry - we'll start with simple concepts that build your confidence step by step.

Coming Up: Basic Programming Concepts

- Programming = giving detailed instructions (like a recipe)
- Variables, commands, and procedures in plain English
- Reading code before writing code
- Making small modifications to existing models

Preparation for next time: Think about a social phenomenon you'd like to model. What individual behaviors might create interesting collective patterns?

4.g. Basic Programming Concepts

Now we start the journey towards telling computers what to do. Don't be afraid - programming is just giving detailed instructions! This tutorial builds your confidence with NetLogo's English-like commands.

Figure 9: Overview of NetLogo programming language commands and primitives.

Note

Time Required: 50 minutes
Prerequisites: Completed NetLogo Introduction & Interface

4.g.i. *Programming Without Fear (10 min):*

Let's start with a fundamental truth: **Programming is just giving detailed instructions to a computer.**

Programming = Giving Detailed Instructions:

The only difference: computers need **very precise** instructions and can't fill in missing details like humans can.

Like Writing a Recipe or Assembly Instructions

Think about IKEA furniture instructions - they're extremely detailed because they assume you know nothing. Programming is the same way!

NetLogo Uses English-like Commands:

Unlike many programming languages, NetLogo reads almost like English:

- `create-turtles 50` (create 50 turtles)
- `forward 1` (move forward 1 step)
- `set color red` (change color to red)
- `count neighbors` (how many neighbors do I have?)

You can see most of the commands in the Figure 1.

Also, Netlogo provides a dictionary of commands in their website: NetLogo Dictionary.

Save this page in your bookmarks for easy reference!

Figure 10: NetLogo Commands Word Cloud

4.g.ii. *Core Concepts (25 min):*

Let's learn the basic building blocks that all programming uses.

Variables: Things That Can Change:

Variables store information that can change over time.:

Examples in real life:

- Your age (changes every year)
- Your bank account balance (goes up and down)
- Your location (changes as you move)

Examples in NetLogo:

- `age` - how old is this turtle?
- `wealth` - how much money does this turtle have?
- `xcor` - what is the turtle's x-coordinate?
- `color` - what color is this turtle?

Think of Variables as Labels:

Imagine each turtle wearing name tags:

Commands: Actions Agents Can Take:

Commands tell agents what to do.

Basic movement commands:

- `forward 1` - move forward 1 step
- `right 90` - turn right 90 degrees
- `left 45` - turn left 45 degrees

Property change commands:

- `set color red` - change color to red
- `set size 2` - make turtle bigger
- `die` - remove this turtle from the world

Social commands::

- `create-link-with turtle 5` - form connection with turtle #5
- `ask neighbors` - give instructions to nearby turtles

Reporters: Questions Agents Can Answer:

Reporters ask questions and get answers.:

About myself:

- `who` - what is my ID number?
- `xcor` - what is my x-coordinate?

- `count my-links` - how many connections do I have?

About others:

- `count turtles` - how many turtles exist?
- `count neighbors` - how many turtles are near me?
- `mean [wealth] of turtles` - what's the average wealth?

About the environment:

- `pcolor` - what color is the patch I'm on?
- `patches in-radius 3` - which patches are within 3 units?

Procedures: Grouping Instructions Together:

Procedures are like recipes - they group related instructions.

```
to move-randomly
  right random 360    ; turn a random amount
  forward 1           ; move forward 1 step
end
```

Program 1: Simple Procedure Example

This procedure called `move-randomly` does two things:

1. Turn a random direction (0-360 degrees)
2. Move forward 1 step

Now you can just say `move-randomly` instead of repeating those two lines!

Why use procedures?:

- Organize related instructions
- Avoid repeating the same code
- Make code easier to read and understand
- Break complex tasks into smaller pieces

4.g.iii. *Reading Code Before Writing (15 min):*

Let's practice reading and understanding NetLogo code together.

Activity 1: Code Reading:

Look at this simple procedure. What do you think it does?

```
to setup
  clear-all
  create-turtles 100 [
    setxy random-xcor random-ycor
    set color one-of [red green blue yellow]
  ]
  reset-ticks
end
```

Activity 2: Spot the Difference:

Compare these two procedures. What's different?

Procedure A:

```
to move-turtles
  ask turtles [
    forward 1
  ]
end
```

Procedure B:

```
to move-turtles
  ask turtles [
    right random 360
    forward 1
  ]
end
```

What's the difference in behavior? TTYN

Activity 3: Human Computer:

Let's act out NetLogo commands!

Setup:

- 5 students are "turtles"
- Classroom is the "world"
- One student is the "computer" giving commands

Commands to try:

1. `create-turtles 5` - 5 students enter the "world"
2. `ask turtles [ forward 2 ]` - everyone takes 2 steps forward

3. `ask turtles [ right 90 ]` - everyone turns right
4. `ask turtle 0 [ set color red ]` - student #0 puts on red hat
5. `ask turtles [ if color = red [ forward 3 ] ]` - only red turtle moves

4.g.iv. *What We've Accomplished:*

Learning Objectives Achieved:

By completing this tutorial, you now:

✓ Understand that code is just detailed instructions
✓ Recognize basic programming concepts in NetLogo syntax
✓ Can read and interpret simple NetLogo procedures
✓ Feel prepared to modify existing code

What's Next?:

Now that you understand the basic building blocks, we'll learn how to work with individual agents (turtles) - creating them, controlling them, and making them interact.

Think about: What kinds of agents would you want in a model of your research interest? What properties would they need? What behaviors?

4.h. Agents

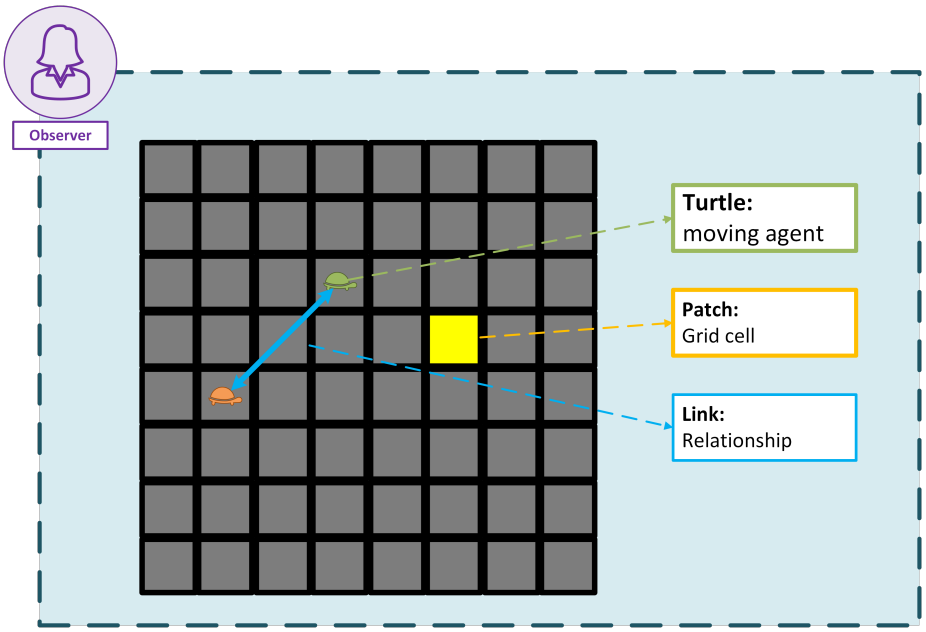
Agents are the main actors in NetLogo. They can be **turtles**, **patches**, **links**, or the **observer**. Each agent has its own set of properties and behaviors.

The **turtles** are the most common type of agent. They can move around the **world**, interact with other **turtles**, and change their properties. **Turtles** can also have different shapes and colors, which can be used to represent different types of agents in your model.

The **world** is made up of **patches**, which are the individual squares that make up the 2D grid. Each **patch** can have its own properties, such as color and elevation. **Patches** can also be used to represent different types of terrain or resources in your model. The **turtles** can move around the **patches** and interact with them, which allows you to create complex models of behavior and interaction.

Links are a type of agents used to connect **turtles** and represent relationships between them. They can be used to represent social networks, transportation systems, or any other type of connection between agents.

The **observer** is a special agent that can control the simulation and interact with the other agents. The **observer** can create and destroy **turtles**, change the properties of **patches**, and control the flow of the simulation. The **observer** can also be used to create user interfaces, such as buttons and sliders, to control the simulation. It is important to note that the **observer** does not have any properties or behaviors of its own, but it can interact with all other agents in the simulation.



Note

> When NetLogo is initiate, there are no **turtles** in the **world**. The observer can create **turtles** using the `create-turtles` command. **Patches** can also create **turtles** using the `sprout` command.
>
> Patches can't move, but you can consider them as active agents from the beggining.

4.h.i. *Working with Agents (Turtles):*

Time to meet the stars of your models - the individual agents! In NetLogo, these are called “turtles” and they represent people, animals, organizations, or any individual actors.

> **Note**
>
> **Time Required:** 60 minutes
> **Prerequisites:** Basic Programming Concepts

4.h.ii. *Meet the Turtles (10 min):*

Turtles = Individual Agents in Your Model:

In NetLogo, **turtles represent the individual actors** in whatever social system you're studying:

Examples:

- **Sociology:** People in a neighborhood, members of an organization
- **Economics:** Individual traders, consumers, firms
- **Political Science:** Voters, political parties, countries
- **Psychology:** Individual decision-makers with different cognitive styles

Each Turtle Has Properties:

Just like real people have characteristics, each turtle can have properties:

Built-in properties (every turtle has these):

- `color` - what color is this turtle?
- `size` - how big should this turtle appear?
- `xcor` and `ycor` - where is this turtle located?
- `heading` - which direction is this turtle facing?
- `who` - what is this turtle's unique ID number?

Custom properties (you define these):

- `age` - how old is this person?
- `wealth` - how much money does this person have?
- `opinion` - what does this person believe about an issue?
- `social-connections` - how many friends does this person have?

Turtles Can Move, Interact, Make Decisions:

Turtles aren't just static dots - they can:

- **Move** around the world following rules
- **Interact** with other nearby turtles
- **Make decisions** based on their situation
- **Change** their properties over time
- **Remember** things that happened to them

4.h.iii. *Creating and Controlling Turtles (20 min):*

Let's learn how to bring agents to life and give them instructions.

Bringing Agents to Life: `create-turtles`:

The most basic command for creating a population:

```
create-turtles 100
```

This creates 100 turtles, each with:

- Random color
- Random location
- Random heading (direction)
- Size 1, ID numbers 0-99

Create specific numbers:

```
create-turtles 50    ; Create 50 turtles
create-turtles 200   ; Create 200 turtles
```

Giving Instructions to All or Some Agents: `ask turtles`:

The `ask` command lets you give instructions to turtles:

Ask all turtles to do something:

```
ask turtles [
  forward 1           ; Every turtle moves forward
  set color red       ; Every turtle turns red
]
```

Ask specific turtles to do something:

```
ask turtle 0 [         ; Ask turtle #0 specifically
  set color blue
  set size 3
]

ask turtles with [color = red] [    ; Ask only red turtles
  forward 2
]

ask n-of 10 turtles [  ; Ask 10 randomly chosen turtles
  set color green
]
```

Basic Turtle Commands:

Movement commands:

- `forward 1` - move forward 1 step
- `right 90` - turn right 90 degrees

- `left 45` - turn left 45 degrees
- `back 1` - move backward 1 step

Property commands:

- `set color red` - change color to red
- `set size 2` - make turtle twice as big
- `set heading 0` - face north (heading 0)

Try it yourself:

```
create-turtles 20 [
  setxy random-xcor random-ycor    ; Random position
  set color one-of [red blue green yellow]  ; Random color
]

ask turtles [
  right random 360    ; Turn random direction
  forward 3           ; Move forward
]
```

Activity 1: Build a Flock:

Goal: Create turtles that move in the same direction

Step 1: Create the turtles

```
create-turtles 50 [
  setxy random-xcor random-ycor
  set color yellow
  set heading 90  ; All face east
]
```

Step 2: Make them move together

```
ask turtles [
  forward 1
]
```

Step 3: Run this repeatedly and watch your flock move across the screen!

4.h.iv. *Making Turtles Unique (15 min):*

Real people aren't identical - let's give our turtles individual differences.

Giving Turtles Different Properties:

Random assignment:

```
create-turtles 100 [
  set age 18 + random 62        ; Age between 18-79
  set wealth random 10000       ; Wealth between $0-$9,999
  set social-activity random 11 ; Social activity 0-10
]
```

Systematic assignment:

```
create-turtles 100 [
  set age 20 + who / 2          ; Age increases with ID
  if who < 50 [ set color red ] ; First 50 are red
  if who >= 50 [ set color blue ] ; Last 50 are blue
]
```

Using probability distributions:

```
create-turtles 100 [
  ; Most people are middle-aged, few are very young or old
  set age 20 + random-normal 25 10

  ; 70% are cooperative, 30% are competitive
  if random 100 < 70 [ set strategy "cooperative" ]
  if random 100 >= 70 [ set strategy "competitive" ]
]
```

Random vs. Systematic Assignment:

Random assignment creates natural variation:

- `random 10` gives numbers 0-9 randomly
- `one-of [red green blue]` picks one color randomly
- `random-normal 50 15` gives normally distributed numbers around 50

Systematic assignment creates patterns:

- Assign properties based on `who` number
- Create spatial gradients
- Assign roles or types deliberately

Using who Numbers to Identify Specific Turtles:

Every turtle has a unique `who` number (0, 1, 2, 3, ...):

```
ask turtle 0 [ set color red ]        ; Color turtle #0 red
ask turtle 5 [ forward 10 ]           ; Move turtle #5 forward
```

```
ask turtles with [who < 10] [ ... ]   ; First 10 turtles
ask turtles with [who mod 2 = 0] [ ... ] ; Even-numbered turtles
```

Activity 2: Random Walk:

Goal: Make turtles explore randomly

```
to setup
  clear-all
  create-turtles 20 [
    setxy random-xcor random-ycor
    set color one-of [red green blue yellow orange]
    set size 1 + random 2  ; Size between 1-3
  ]
end

to go
  ask turtles [
    right random 360       ; Turn random direction
    forward 1              ; Move forward
    if xcor > max-pxcor or xcor < min-pxcor [
      set xcor random-xcor ; Wrap around edges
    ]
    if ycor > max-pycor or ycor < min-pycor [
      set ycor random-ycor ; Wrap around edges
    ]
  ]
end
```

Run this: Click “setup” once, then click “go” repeatedly to watch random exploration.

4.h.v. *Turtle Interactions (15 min):*

The real power comes when turtles interact with each other.

How Turtles "See" Other Turtles Nearby:

Find neighbors within a distance:

```
ask turtles [
  let nearby-turtles other turtles in-radius 3
  if any? nearby-turtles [
    ; Do something with nearby turtles
  ]
]
```

Find the closest turtle:

```
ask turtles [
  let closest min-one-of other turtles [distance myself]
  if closest != nobody [
    face closest  ; Turn toward closest turtle
  ]
]
```

Simple Interaction Rules:

Follow your neighbors:

```
ask turtles [
  let neighbors other turtles in-radius 3
  if any? neighbors [
    let average-heading mean [heading] of neighbors
    set heading average-heading
  ]
  forward 1
]
```

Avoid crowding:

```
ask turtles [
  let too-close other turtles in-radius 1
  if any? too-close [
    let repel-direction mean [heading] of too-close + 180
    set heading repel-direction
  ]
  forward 1
]
```

Collective Behavior from Individual Actions:

When individual turtles follow simple rules about their neighbors, amazing collective patterns emerge:

- **Flocking:** Turtles that align with neighbors create flocks

- **Segregation:** Turtles that prefer similar neighbors create clusters
- **Information spread:** Turtles that copy neighbors spread information
- **Traffic jams:** Turtles that slow down when crowded create jams

Activity 3: Color Copying:

Goal: Turtles change color to match nearby turtles

```
to setup
  clear-all
  create-turtles 100 [
    setxy random-xcor random-ycor
    set color one-of [red green blue]
  ]
end

to go
  ask turtles [
    ; Look at nearby turtles
    let neighbors other turtles in-radius 2

    if any? neighbors [
      ; Find the most common color among neighbors
      let red-neighbors neighbors with [color = red]
      let green-neighbors neighbors with [color = green]
      let blue-neighbors neighbors with [color = blue]

      ; Change to majority color
      if count red-neighbors > count green-neighbors and
         count red-neighbors > count blue-neighbors [
        set color red
      ]
      if count green-neighbors > count red-neighbors and
         count green-neighbors > count blue-neighbors [
        set color green
      ]
      if count blue-neighbors > count red-neighbors and
         count blue-neighbors > count green-neighbors [
        set color blue
      ]
    ]

    ; Move randomly
    right random 60 - 30
    forward 0.5
  ]
end
```

Run this model:

1. Click “setup” to create mixed-color turtles
2. Click “go” repeatedly and watch colors cluster together

3. Notice how local copying creates global patterns!

4.h.vi. *What We've Accomplished:*

Learning Objectives Achieved:

By completing this tutorial, you can now:

✓ Create and control basic turtle agents
✓ Understand how individual agent rules create collective patterns
✓ Modify turtle properties and behaviors
✓ Implement simple agent interactions

What's Next?:

Now that you can work with agents, let's explore their environment! Agents don't exist in a vacuum - they live in a world that shapes their behavior.

Think about: In your research area, what kind of environment do agents live in? Physical space? Social networks? Economic markets? Information landscapes?

4.i. Environment and Patches

Agents don't exist in empty space - they live in an environment that shapes their behavior. In NetLogo, the environment is made up of "patches" that can represent anything from physical locations to abstract spaces.

> **Note**
>
> **Time Required:** 40 minutes
> **Prerequisites:** Working with Agents (Turtles)

4.i.i. *The World Patches Live In (10 min):*

Patches = Grid Squares That Make Up the World:

Think of the NetLogo world as a **grid of squares**, like:

- **Chessboard squares** where pieces can move
- **City blocks** where people live and work
- **Pixels on a screen** that create an image
- **Cells in a spreadsheet** that hold information

Each square is called a **patch** and has:

- **Coordinates:** `pxcor` (x-position) and `pycor` (y-position)
- **Color:** `pcolor` (what color is this patch?)
- **Custom properties:** anything you define (temperature, resources, ownership, etc.)

Each Patch Has Properties (Color, Variables):

Built-in properties:

- `pxcor` and `pycor` - location coordinates
- `pcolor` - color of this patch
- `plabel` - text label on this patch

Custom properties you might add:

- `temperature` - how hot/cold is this location?
- `resources` - how much food/oil/wealth is here?
- `population-density` - how crowded is this area?
- `pollution-level` - how contaminated is this spot?

Environment Shapes Agent Behavior:

The environment isn't just decoration - it **actively influences** what agents do:

Examples:

- **Foraging:** Animals move toward resource-rich patches
- **Urban planning:** People prefer low-pollution, high-amenity areas
- **Disease spread:** Infection rates vary by population density
- **Economic development:** Businesses locate near transportation hubs

4.i.ii. *Patch Properties and Visualization (15 min):*

Let's learn how to use patch colors to visualize data and create meaningful environments.

Using Patch Color to Show Data:

Basic color assignment:

```
ask patches [
  set pcolor green    ; All patches green
]

ask patch 0 0 [       ; Center patch only
  set pcolor red
]

ask patches with [pxcor > 0] [  ; Right half of world
  set pcolor blue
]
```

Color based on data:

```
ask patches [
  ; Color based on distance from center
  let distance-from-center sqrt (pxcor ^ 2 + pycor ^ 2)
  set pcolor scale-color red distance-from-center 0 10
]
```

The `scale-color` command creates gradients:

- `scale-color red value 0 10` makes a red gradient from light (value=0) to dark (value=10)
- High values = dark red, low values = light red

Creating Environmental Gradients:

Temperature gradient (hot in south, cold in north):

```
ask patches [
  set temperature pycor + 10        ; Temperature based on y-coordinate
  set pcolor scale-color red temperature 0 20  ; Red = hot, pink = cold
]
```

Resource distribution (rich center, poor edges):

```
ask patches [
  let distance-from-center sqrt (pxcor ^ 2 + pycor ^ 2)
  set resources 100 - distance-from-center * 5
  if resources < 0 [ set resources 0 ]
  set pcolor scale-color green resources 0 100  ; Green = rich, black =
poor
]
```

Random patchy resources:

```
ask patches [
  set resources random 100
  set pcolor scale-color brown resources 0 100  ; Brown gradient
]
```

Activity 1: Heat Map:

Goal: Create a temperature gradient using patch colors

```
to setup-heat-map
  ask patches [
    ; Create temperature based on distance from center
    let distance-from-center sqrt (pxcor ^ 2 + pycor ^ 2)
    set temperature 100 - distance-from-center * 3
    if temperature < 0 [ set temperature 0 ]

    ; Color patches based on temperature
    set pcolor scale-color red temperature 0 100
  ]
end
```

Try different patterns:

- East-west gradient: `set temperature pxcor + 15`
- Diagonal gradient: `set temperature (pxcor + pycor) + 20`
- Multiple hot spots: Create several high-temperature centers

4.i.iii. *Turtle-Patch Interactions (15 min):*

Now let's make agents interact with their environment in meaningful ways.

Turtles Asking Their Current Patch Questions:

Turtles can ask the patch they're standing on for information:

```
ask turtles [
  let current-temp [temperature] of patch-here
  if current-temp > 50 [
    set color red      ; Turn red if on hot patch
  ]
]
```

Common patch queries:

- `[pcolor] of patch-here` - what color is this patch?
- `[resources] of patch-here` - how many resources here?
- `[temperature] of patch-here` - what's the temperature?

Moving Based on Patch Properties:

Move toward better patches:

```
ask turtles [
  ; Look at nearby patches
  let nearby-patches patches in-radius 2
  let best-patch max-one-of nearby-patches [resources]

  if best-patch != nobody [
    face best-patch     ; Turn toward resource-rich patch
    forward 1
  ]
]
```

Avoid dangerous areas:

```
ask turtles [
  let current-pollution [pollution] of patch-here
  if current-pollution > 50 [
    ; Move away from polluted areas
    let clean-patches patches in-radius 3 with [pollution < 10]
    if any? clean-patches [
      move-to one-of clean-patches
    ]
  ]
]
```

Turtles Modifying Their Environment:

Agents don't just respond to environment - they change it:

Consume resources:

```
ask turtles [
  let current-resources [resources] of patch-here
  if current-resources > 0 [
    ask patch-here [
      set resources resources - 1     ; Consume 1 unit
      set pcolor scale-color green resources 0 100  ; Update color
    ]
  ]
]
```

Leave traces:

```
ask turtles [
  ask patch-here [
    set pheromone pheromone + 1       ; Leave pheromone trail
    set pcolor scale-color yellow pheromone 0 10
  ]
]
```

Activity 2: Foraging:

Goal: Turtles move toward resource-rich patches

```
to setup-foraging
  ; Create resource patches
  ask patches [
    set resources random 50
    set pcolor scale-color green resources 0 50
  ]

  ; Create foraging turtles
  create-turtles 20 [
    setxy random-xcor random-ycor
    set color yellow
    set energy 100
  ]
end

to go-foraging
  ask turtles [
    ; Look for nearby resource patches
    let nearby-patches patches in-radius 2
    let best-patch max-one-of nearby-patches [resources]

    if best-patch != nobody [
      face best-patch
      forward 1
    ]

    ; Consume resources on current patch
    let current-resources [resources] of patch-here
    if current-resources > 0 [
      set energy energy + current-resources
```

```
      ask patch-here [
        set resources 0
        set pcolor black  ; Mark as depleted
      ]
    ]

    ; Use energy to survive
    set energy energy - 1
    if energy <= 0 [ die ]
  ]
end
```

Run this model:

1. Click "setup-foraging" to create resources and turtles
2. Click "go-foraging" repeatedly and watch turtles search for food
3. Notice how they deplete resources and change the environment

Activity 3: Erosion:

Goal: Turtles modify patch values as they pass through

```
to setup-erosion
  ; Create terrain with different soil stability
  ask patches [
    set soil-stability 50 + random 50  ; Stability 50-100
    set pcolor scale-color brown soil-stability 50 100
  ]

  create-turtles 30 [
    setxy random-xcor random-ycor
    set color white
  ]
end

to go-erosion
  ask turtles [
    ; Random movement
    right random 60 - 30
    forward 1

    ; Cause erosion on current patch
    ask patch-here [
      set soil-stability soil-stability - 0.5
      if soil-stability < 0 [ set soil-stability 0 ]
      set pcolor scale-color brown soil-stability 0 100
    ]
  ]
end
```

What happens: Watch how turtle movement creates "erosion paths" in the landscape!

Learning Objectives Achieved:

By completing this tutorial, you can now:

✓ **Understand the role of environment in agent-based models**
✓ **Create meaningful environmental visualizations**
✓ **Implement agent-environment interactions**

What's Next?:

Now you have all the building blocks - agents, environment, and interactions. Time to put it all together into your first complete model from scratch!

Think about: What complete model would you like to build? What social phenomenon interests you? What agents and environment would you need?

4.i.iv. *What We've Accomplished:*

Learning Objectives Achieved:

By completing this tutorial, you can now:

✓ Understand the role of environment in agent-based models
✓ Create meaningful environmental visualizations
✓ Implement agent-environment interactions

What's Next?:

Now you have all the building blocks - agents, environment, and interactions. Time to put it all together into your first complete model from scratch!

Think about: What complete model would you like to build? What social phenomenon interests you? What agents and environment would you need?

4.j. Building Your First Complete Model

Time to put everything together! You'll plan and build a complete model from scratch using all the skills you've learned.

> **Note**
>
> **Time Required:** 55 minutes
> **Prerequisites:** Environment and Patches

4.j.i. *Planning Before Programming (15 min):*

Good models start with good plans. Let's think through the process systematically.

What Question Are You Trying to Answer?:

Every model should address a specific question about social phenomena:

Good research questions:

- "How do individual preferences for similar neighbors lead to residential segregation?"
- "What factors determine whether cooperation emerges in a group?"
- "How does information spread through a social network?"

Too vague:

- "How do people behave?"
- "What makes societies work?"

What Are Your Agents and What Do They Do?:

Once you have a question, design your agents:

Agent properties: What characteristics do they have?

- Demographics (age, income, education)
- Preferences (risk tolerance, social orientation)
- States (opinion, mood, health status)
- Resources (money, time, information)

Agent behaviors: What actions can they take?

- Movement (where do they go?)
- Communication (who do they talk to?)
- Decision-making (how do they choose?)
- Learning (how do they adapt?)

What Does Success Look Like?:

Define what patterns you expect to see:

- **Segregation model:** Clusters of similar agents
- **Cooperation model:** Stable cooperation despite temptation to defect
- **Information spread:** Rapid diffusion through social networks

Success criteria:

- Model produces realistic patterns
- Changing parameters creates predictable changes
- Results provide insights about real-world phenomena

4.j.ii. *Step-by-Step Model Building (30 min):*

Let's build a complete model together using systematic steps.

Our Project: Ants Following Pheromone Trails:

Research question: "How do individual ants create efficient collective foraging paths?"

Agents: Ants that search for food and leave pheromone trails **Environment:** Food sources and evaporating pheromone trails **Behavior:** Ants follow pheromone gradients and reinforce successful paths

Start Simple: Create Agents:

Step 1: Basic setup

```
turtles-own [
  carrying-food?    ; Is this ant carrying food back to nest?
]

to setup
  clear-all

  ; Create nest at center
  ask patch 0 0 [
    set pcolor brown
    set plabel "NEST"
  ]

  ; Create food source
  ask patches with [pxcor > 10 and pycor > 10] [
    if random 100 < 30 [  ; 30% chance of food
      set pcolor green
    ]
  ]

  ; Create ants
  create-turtles 50 [
    setxy 0 0           ; Start at nest
    set color red
    set carrying-food? false
  ]

  reset-ticks
end
```

Test it: Run setup and verify you see nest, food, and ants.

Add One Behavior at a Time:

Step 2: Basic movement

```
to go
  ask turtles [
    ; Simple random movement for now
    right random 60 - 30    ; Turn randomly
    forward 1
  ]
  tick
end
```

Test it: Run go repeatedly. Do ants move around randomly?

Step 3: Food collection

```
to go
  ask turtles [
    ; Check if on food patch
    if pcolor = green and not carrying-food? [
      set carrying-food? true
      set color yellow        ; Carrying food
      ask patch-here [ set pcolor black ]  ; Remove food
    ]

    ; Check if back at nest with food
    if pcolor = brown and carrying-food? [
      set carrying-food? false
      set color red           ; Not carrying food
    ]

    ; Movement
    right random 60 - 30
    forward 1
  ]
  tick
end
```

Test it: Do ants pick up food and return to nest?

Test Frequently, Fix Problems Early:

After each step, ask:

- Does the behavior work as expected?
- Are there any error messages?
- Do you see the visual changes you expect?

Common issues:

- Ants getting stuck at world edges
- Food disappearing too quickly
- Ants not finding their way back to nest

Build Complexity Gradually:

Step 4: Add pheromone trails

```
patches-own [
  pheromone          ; Amount of pheromone on this patch
]

to setup
  ; ... previous setup code ...

  ; Initialize pheromones
  ask patches [
    set pheromone 0
  ]
end

to go
  ask turtles [
    ; Leave pheromone if carrying food
    if carrying-food? [
      ask patch-here [
        set pheromone pheromone + 10
        set pcolor scale-color red pheromone 0 100
      ]
    ]

    ; ... previous behavior code ...
  ]

  ; Evaporate pheromones
  ask patches [
    set pheromone pheromone * 0.95  ; 5% evaporation
    if pheromone < 0.1 [ set pheromone 0 ]
    if pcolor != brown and pcolor != green [
      set pcolor scale-color red pheromone 0 100
    ]
  ]

  tick
end
```

Test it: Do you see red trails where ants have been?

Step 5: Follow pheromone gradients

```
to go
  ask turtles [
    ; If not carrying food, follow pheromone gradients
    if not carrying-food? [
      let best-patch max-one-of patches in-radius 2 [pheromone]
      if best-patch != nobody and [pheromone] of best-patch > 0 [
        face best-patch
        forward 1
```

```
      ] else [
        right random 60 - 30  ; Random movement if no pheromone
        forward 1
      ]
    ] else [
      ; If carrying food, head toward nest
      face patch 0 0
      forward 1
    ]

    ; ... food collection code ...
    ; ... pheromone laying code ...
  ]

  ; ... pheromone evaporation code ...
  tick
end
```

Activity: Mini-Project:

Complete the ant model by adding:

1. **Better nest-finding:** Ants carrying food should move more directly toward nest
2. **Trail reinforcement:** Successful ants should lay stronger pheromone trails
3. **Energy system:** Ants use energy and must return to nest to refuel

Model Building Tips:

Start simple, add complexity gradually:

- Get basic movement working first
- Add one new behavior at a time
- Test after every change
- Fix problems immediately

Use meaningful variable names:

- `carrying-food?` not `cf?`
- `pheromone-strength` not `ps`
- `energy-level` not `el`

Add comments explaining what code does:

```
; Ants lay pheromone when carrying food
if carrying-food? [
  ask patch-here [
    set pheromone pheromone + 10
  ]
]
```

4.j.iii. *Documentation and Sharing (10 min):*

Good models need clear documentation so others can understand and use them.

Writing Clear Comments in Code:

Comment every major section:

```
to go
  ; MOVEMENT PHASE: Ants move based on pheromone trails
  ask turtles [
    if not carrying-food? [
      follow-pheromone-trail
    ] else [
      return-to-nest
    ]
  ]

  ; ENVIRONMENT PHASE: Update pheromone trails
  evaporate-pheromones

  tick
end
```

Explain complex calculations:

```
; Calculate pheromone gradient (difference between current and best
patch)
let current-pheromone [pheromone] of patch-here
let best-nearby max [pheromone] of patches in-radius 2
let gradient best-nearby - current-pheromone
```

Using the Info Tab Effectively:

Include these sections in your Info tab:

```
**WHAT IS IT?**
Brief description of the phenomenon and research question

**HOW IT WORKS**
Explanation of agent behaviors and interactions

**HOW TO USE IT**
Instructions for running the model and interpreting results

**THINGS TO NOTICE**
Key patterns to watch for

**THINGS TO TRY**
Suggested experiments with different parameters
```

Preparing Models for Others to Understand:

Model Documentation Checklist

Code Quality:

- Clear, meaningful variable names
- Comments explaining complex sections
- Organized into logical procedures

User Interface:

- Sliders for key parameters
- Buttons for setup and go
- Monitors showing important statistics
- Plots tracking key variables over time

Documentation:

- Complete Info tab
- Clear instructions
- Background information and references

Testing:

- Model runs without errors
- Results are reasonable and interesting
- Different parameter values produce different outcomes

Activity: Code Review:

Pairs review each other's models:

1. **Can you understand what the model does** without explanation?
2. **Are variable names clear and meaningful?**
3. **Are there enough comments** to follow the logic?
4. **Does the Info tab explain** how to use the model?
5. **What questions does the model raise** that could be explored further?

4.j.iv. *What We've Accomplished:*

Learning Objectives Achieved:

By completing this tutorial, you can now:

✓ **Plan and implement a complete simple model**
✓ **Use iterative development process**
✓ **Document models clearly for others**
✓ **Debug common problems**

What's Next?:

You can now build complete models! Next we'll learn how to collect meaningful data from your models and analyze the results systematically.

Congratulations! You've built your first complete agent-based model from scratch. This is a significant achievement that many researchers never accomplish.

4.k. Data Collection and Analysis

Every good model tells us something about the world. To extract meaningful insights, you need to collect and analyze data systematically.

> **Note**
>
> **Time Required:** 45 minutes
> **Prerequisites:** Building Your First Complete Model

4.k.i. *Why Data Matters in Modeling (10 min):*

Models without data are just entertaining animations. To answer research questions, you need systematic measurement and analysis.

Models Answer Questions with Evidence:

Without data: "My segregation model seems to create clusters sometimes." **With data:** "When tolerance is below 30%, segregation emerges in 95% of runs within 200 time steps."

Without data: "Cooperation appears to work in this model." **With data:** "Cooperation is stable when the population is smaller than 50 agents, but collapses above 75 agents."

What Makes Good Model Data?:

Systematic measurement:

- Track the same variables consistently
- Measure at regular intervals
- Run multiple times to account for randomness
- Test different parameter values systematically

Meaningful variables:

- **Outcome measures:** What you're trying to explain (segregation level, cooperation rate, infection spread)
- **Input parameters:** What you're varying (tolerance levels, payoff values, transmission rates)
- **Process indicators:** How things change over time (movement patterns, network formation, learning rates)

Activity: Identifying Key Variables:

4.k.ii. *Using NetLogo's Built-in Analysis Tools (20 min):*

NetLogo provides powerful tools for data collection without programming complex analysis routines.

Monitors: Real-Time Data Tracking:

Monitors show live values as your model runs:

```
; In interface, create monitor that shows:
count turtles with [carrying-food?]
; Label it: "Ants with Food"

mean [pheromone] of patches with [pheromone > 0]
; Label it: "Average Pheromone"

ticks
; Label it: "Time Steps"
```

Strategic monitor placement:

- **Outcome variables:** Your main research question measures
- **Sanity checks:** Total population, conservation laws
- **Process indicators:** Rates of change, intermediate states

Plots: Visualizing Change Over Time:

Plots track multiple variables and show patterns:

Basic setup in Interface tab:

1. Click "Plot" button to create new plot
2. Name it (e.g., "Population Dynamics")
3. Set X and Y axis labels and ranges
4. Add "pens" for different variables

Code to update plots:

```
to update-plots
  ; This automatically runs every tick

  ; Plot 1: Population by type
  set-current-plot "Population Dynamics"
  set-current-plot-pen "Cooperators"
  plot count turtles with [strategy = "cooperate"]

  set-current-plot-pen "Defectors"
  plot count turtles with [strategy = "defect"]

  ; Plot 2: Average satisfaction
  set-current-plot "Satisfaction Levels"
  set-current-plot-pen "satisfaction"
```

```
  plot mean [satisfaction] of turtles
end
```

Histograms: Understanding Distributions:

See how values are distributed across your population:

```
; In interface, create histogram
; Set pen update to:
histogram [age] of turtles

; Or show distribution of some other property:
histogram [count link-neighbors] of turtles  ; Network degree
histogram [money] of turtles                  ; Wealth distribution
histogram [satisfaction] of turtles           ; Satisfaction levels
```

Activity: Build Analysis Dashboard:

Create a complete monitoring system:

1. **Add 3-4 monitors** tracking key variables in your model
2. **Create 2 plots** showing how important things change over time
3. **Add 1 histogram** showing distribution of agent properties
4. **Run your model and observe** what patterns emerge

BehaviorSpace: Systematic Experiments:

BehaviorSpace runs multiple experiments automatically with different parameter combinations.

Setting up experiments:

1. Tools menu arrow.r BehaviorSpace
2. Click “New” to create experiment
3. Vary parameters systematically:

```
["tolerance" [0.1 0.2 0.3 0.4 0.5]]
["population-size" [100 200 500]]
```

4. Set number of repetitions (e.g., 10 runs per combination)
5. Choose which variables to measure:

```
count turtles with [color = red]
segregation-index
mean [satisfaction] of turtles
```

6. Run experiment (can take a while!)

BehaviorSpace generates spreadsheet with all results for further analysis.

4.k.iii. *Exporting Data for External Analysis (15 min):*

Sometimes you need more sophisticated analysis than NetLogo provides. Exporting data lets you use Excel, R, Python, or other tools.

File Output Commands:

Write data to CSV files that other programs can read:

```
; Open file for writing (do this in setup)
file-open "model-output.csv"
file-print "tick,cooperators,defectors,mean-payoff"
file-close

; Write data each tick (do this in go procedure)
file-open "model-output.csv"
file-print (word ticks ","
                 count turtles with [strategy = "cooperate"] ","
                 count turtles with [strategy = "defect"] ","
                 mean [payoff] of turtles)
file-close
```

Better approach using procedures:

```
to setup
  ; ... other setup code ...
  setup-output-file
end

to setup-output-file
  ; Delete old file and create new one with headers
  if file-exists? "results.csv" [ file-delete "results.csv" ]
  file-open "results.csv"
  file-print "tick,cooperation_rate,mean_payoff,clustering"
  file-close
end

to record-data
  file-open "results.csv"
  let coop-rate count turtles with [strategy = "cooperate"] / count
turtles
  let avg-payoff mean [payoff] of turtles
  let clustering-measure calculate-clustering  ; Your custom measure

  file-print (word ticks "," coop-rate "," avg-payoff "," clustering-
measure)
  file-close
end

to go
  ; ... model update code ...
  record-data  ; Call this every tick or every N ticks
```

```
  tick
end
```

Network Analysis Output:

Export network structure for analysis in specialized tools:

```
to export-network
  ; Export edge list
  file-open "network-edges.csv"
  file-print "source,target,weight"
  ask links [
    file-print (word [who] of end1 "," [who] of end2 "," link-weight)
  ]
  file-close

  ; Export node attributes
  file-open "network-nodes.csv"
  file-print "id,strategy,payoff,degree"
  ask turtles [
    file-print (word who "," strategy "," payoff "," count link-
neighbors)
  ]
  file-close
end
```

Spatial Data Output:

Export spatial patterns for GIS or mapping analysis:

```
to export-spatial-data
  file-open "spatial-data.csv"
  file-print "x,y,population,wealth,satisfaction"
  ask patches [
    if count turtles-here > 0 [
      file-print (word pxcor "," pycor ","
                       count turtles-here ","
                       mean [wealth] of turtles-here ","
                       mean [satisfaction] of turtles-here)
    ]
  ]
  file-close
end
```

Activity: Data Export Practice:

Choose one type of data export and implement it:

1. **Time series:** Track 3-4 key variables over time
2. **Network data:** Export agent connections and attributes
3. **Spatial data:** Export location-based information
4. **Test your export** by running model and checking output file

Reproducible Research:

Document your analysis process:

1. **Save parameter settings** used for each analysis
2. **Record random seeds** so others can reproduce exact results
3. **Document data processing steps** from model output to final conclusions
4. **Share both model files and analysis scripts**

4.k.iv. *What We've Accomplished:*

Learning Objectives Achieved:

By completing this tutorial, you can now:

✓ Use monitors, plots, and histograms for real-time analysis
✓ Design systematic experiments with BehaviorSpace
✓ Export data for external analysis tools
✓ Create reproducible research workflows

What's Next?:

You now have the tools to extract meaningful insights from your models! Our final tutorial covers advanced techniques and troubleshooting to help you become a confident modeler.

Well done! You can now build models that provide real insights about social phenomena. This analytical capability is what separates serious modelers from casual users.

4.l. Advanced Topics and Troubleshooting

You've learned the fundamentals - now let's tackle advanced techniques and solve common problems that every serious modeler encounters.

> **Note**
>
> **Time Required:** 45 minutes
> **Prerequisites:** Data Collection and Analysis

4.l.i. *Common Problems and Solutions (20 min):*

Every modeler faces challenges. Here are the most common issues and how to resolve them systematically.

Runtime Errors and How to Fix Them:

"I can't reproduce by" error:

```
Problem: ask patches in-radius 5 [ set color red ]
Error: "I can't reproduce by -1.2345"
```

Cause: A patch coordinate calculation produced a non-integer value **Solution:** Use `round` or explicitly create patch coordinates

```
; Problem version:
let target-x xcor + distance * cos heading
ask patch target-x ycor [ set color red ]

; Fixed version:
let target-x round (xcor + distance * cos heading)
ask patch target-x ycor [ set color red ]

; Better version:
let target-patch patch-at (distance * dx) (distance * dy)
if target-patch != nobody [ ask target-patch [ set color red ] ]
```

"Cannot access a property of a turtle/patch/link who doesn't exist" error:

```
; Problem: turtle died but code still tries to use it
ask turtle 5 [ set color red ]  ; What if turtle 5 is dead?

; Solution: Check if turtle exists first
if turtle 5 != nobody [ ask turtle 5 [ set color red ] ]

; Better: Use breeds and with clauses
ask wolves with [energy > 0] [ hunt ]
```

"Runtime error: Division by zero":

```
; Problem:
let average total-wealth / count turtles  ; What if no turtles?

; Solution:
let average 0
if count turtles > 0 [
  set average total-wealth / count turtles
]

; Better: Handle edge cases explicitly
to-report calculate-average [value-list]
  if length value-list = 0 [ report 0 ]
```

```
  report sum value-list / length value-list
end
```

Performance Issues:

Model runs very slowly:

Common causes and fixes:

1. **Too many agents asking too many others:**

```
; Slow version:
ask turtles [
  ask other turtles [  ; N × N operations!
    if distance myself < 5 [ ... ]
  ]
]

; Faster version:
ask turtles [
  ask other turtles in-radius 5 [  ; Only nearby turtles
    ; ... interaction code ...
  ]
]
```

2. **Unnecessary calculations every tick:**

```
; Slow: recalculates every tick
ask turtles [
  let my-neighborhood turtles in-radius 5
  let similar count my-neighborhood with [color = [color] of myself]
  let satisfaction similar / count my-neighborhood
]

; Faster: only recalculate when needed
ask turtles [
  if moved-recently? [  ; Only when something changed
    calculate-satisfaction
    set moved-recently? false
  ]
]
```

3. **Complex patch operations:**

```
; Slow: asking every patch every tick
ask patches [
  set pheromone pheromone * 0.95  ; Even patches with no pheromone
]

; Faster: only patches that need updating
ask patches with [pheromone > 0] [
  set pheromone pheromone * 0.95
  if pheromone < 0.01 [ set pheromone 0 ]
]
```

Logic Errors:

Model produces unrealistic results:

Debugging Strategy:

Step 1: Verify basic mechanics

- Are agents created correctly?
- Do they move as expected?
- Are calculations producing reasonable numbers?

Step 2: Test edge cases

- What happens with 1 agent? 1000 agents?
- What if all agents have the same properties?
- What if parameters are set to extreme values?

Step 3: Add debugging output

```
; Add temporary monitors or print statements
if who = 0 and ticks mod 50 = 0 [  ; Only turtle 0, every 50 ticks
  print (word "Energy: " energy " Satisfaction: " satisfaction)
]
```

Step 4: Simplify temporarily

- Comment out complex behaviors
- Test one mechanism at a time
- Use simple movement patterns first

Activity: Debug Challenge:

Here's a broken segregation model. Find and fix the errors:

```
to setup
  clear-all
  ask patches [
    if random 100 < density [
      sprout 1 [
        set color one-of [red blue]
        set tolerance random-float 1
      ]
    ]
  ]
end

to go
  ask turtles [
    let similar count turtles-here with [color = [color] of myself]
    let total count turtles-here
    if similar / total < tolerance [
      move-to one-of patches with [count turtles-here = 0]
```

```
    ]
  ]
  tick
end
```

Hint: There are at least 3 logical errors in this code.

4.l.ii. *Extending Models with New Features (15 min):*

Once you have a working model, you can add sophisticated features to address new research questions.

Adding Social Networks:

Basic network formation:

```
turtles-own [
  social-connections  ; List of other turtles this one knows
]

to setup
  ; ... basic setup ...
  ask turtles [
    set social-connections []
  ]
  create-social-network
end

to create-social-network
  ; Each turtle connects to nearby turtles
  ask turtles [
    let potential-friends other turtles in-radius 5
    let num-connections min list 5 count potential-friends

    ask n-of num-connections potential-friends [
      ; Create bidirectional social connection
      set social-connections lput myself social-connections
      ask myself [
        set social-connections lput myself social-connections
      ]
    ]
  ]
end

to spread-information
  ask turtles with [has-information?] [
    ; Share with social connections
    foreach social-connections [ friend ->
      ask friend [
        if random 100 < influence-probability [
          set has-information? true
        ]
      ]
    ]
  ]
end
```

Dynamic Environment Changes:

Environmental shocks and changes:

```
to environmental-change
  ; Periodic environmental disruption
  if ticks mod 500 = 0 and ticks > 0 [
    ask n-of (count patches * 0.1) patches [  ; 10% of environment
changes
      set resource-level resource-level * 0.5  ; Resources cut in half
      set pcolor scale-color green resource-level 0 100
    ]
  ]

  ; Gradual climate change
  if ticks mod 100 = 0 [
    ask patches [
      set temperature temperature + 0.1  ; Gradual warming
      update-habitat-suitability
    ]
  ]
end
```

Learning and Adaptation:

Agents that learn from experience:

```
turtles-own [
  strategy          ; Current behavior
  memory           ; List of recent experiences
  success-rate     ; How well current strategy works
]

to adapt-strategy
  ask turtles [
    ; Remember recent outcome
    set memory lput last-payoff memory
    if length memory > 10 [ set memory but-first memory ]  ; Keep only
recent memories

    ; Calculate recent success rate
    if length memory > 5 [
      set success-rate mean memory

      ; Change strategy if doing poorly
      if success-rate < 0.3 [
        set strategy one-of ["cooperate" "defect" "tit-for-tat"]
        set memory []  ; Reset memory with new strategy
      ]
    ]
  ]
end
```

Activity: Feature Addition:

Choose an existing model and add one new feature:

1. **Social influence:** Agents adopt opinions from friends
2. **Environmental change:** Resources or conditions change over time
3. **Learning:** Agents adapt behavior based on success
4. **Heterogeneity:** Different types of agents with different capabilities

4.l.iii. *Where to Go Next in Your Modeling Journey (10 min):*

You now have solid NetLogo skills. Here's how to continue developing as a modeler.

Advanced NetLogo Features:

Extensions for specialized modeling:

- **Network Extension:** Advanced social network analysis
- **GIS Extension:** Real geographic data and spatial analysis
- **R Extension:** Integration with statistical analysis
- **Web Extension:** Online experiments and data collection

Advanced programming techniques:

- **Breeds and inheritance:** Different types of agents with shared behaviors
- **Lists and tables:** Complex data structures for agent memory
- **File I/O:** Reading real-world data into models
- **Behaviorspace automation:** Large-scale systematic experiments

Related Tools and Platforms:

Other agent-based modeling platforms:

- **Mesa (Python):** Code-based modeling with Python libraries
- **MASON (Java):** High-performance modeling for large systems
- **Anylogic:** Commercial platform with GUI design tools
- **Repast:** Research-focused platform with advanced features

Complementary skills:

- **Statistical analysis:** R, Python, or SPSS for data analysis
- **Network analysis:** Gephi, igraph, NetworkX for social networks
- **GIS:** QGIS or ArcGIS for spatial modeling
- **Data visualization:** Tableau, D3.js, or ggplot2 for presenting results

Research and Learning Resources:

Essential readings:

- **"Think Complexity" by Allen Downey:** Complexity science fundamentals
- **"Agent-Based Models" by Nigel Gilbert:** Social science applications
- **NetLogo User Manual:** Complete reference for all features
- **Journal of Artificial Societies and Social Simulation:** Current research

Online communities:

- **NetLogo Users Group:** Active mailing list for questions and discussions
- **Complexity Explorer:** Free courses on complexity science
- **CoMSES (Computational Model Library):** Repository of published models

Final Project Ideas:

Choose a project that excites you:

Social phenomena:

- How do rumors spread through social media?
- What factors determine neighborhood gentrification?
- How do social movements grow and decline?

Economic systems:

- How do markets form and evolve?
- What causes wealth inequality to persist?
- How do innovations diffuse through industries?

Environmental issues:

- How do communities respond to climate change?
- What determines success of conservation efforts?
- How do urban planning decisions affect sustainability?

Organizational behavior:

- How do teams coordinate complex projects?
- What makes some organizations more innovative?
- How do company cultures spread and change?

4.l.iv. *What We've Accomplished:*

Learning Objectives Achieved:

By completing this tutorial series, you can now:

✓ Debug common programming and logic errors
✓ Optimize model performance for large simulations
✓ Extend models with advanced features
✓ Connect NetLogo to broader modeling ecosystem
✓ Plan independent modeling research projects

Congratulations!:

You've completed the NetLogo tutorial series and developed substantial agent-based modeling skills. You can now:

- Build complete models from scratch
- Collect and analyze meaningful data
- Debug problems systematically
- Extend models with new capabilities
- Connect your work to the broader research community

Most importantly: You can use computational modeling to explore questions about social phenomena that interest you. This is a powerful capability that opens doors to research in many fields.

Keep modeling, keep learning, and most importantly - keep asking interesting questions about how social systems work!

4.m. Implementing the Segregation Model

This is a two-week module exploring the Schelling Segregation Model, which demonstrates how individual preferences can lead to large-scale patterns of segregation in neighborhoods. We'll use the NetLogo modeling environment to simulate and analyze these dynamics.

The objective of this mini-tutorial is to implement from scratch a simple version of the Schelling Segregation Model in NetLogo, explore its behavior, and understand its implications for social dynamics.

The instructions below will guide you through building the model step-by-step. After completing the tutorial, you should be able to modify and experiment with the model on your own.

> **Note**
>
> **Estimated Time:** 4-6 hours

4.m.i. *1. Setup Instructions:*

It is always a good practice to start your model by setting up the environment and initializing the agents. In this section, we will create the setup procedure for our segregation model.

To do so, we want to make sure we understand the model we are creating. The intention is to build a grid of agents, where each agent has a color (red or blue) and a preference for the color of its neighbors. If an agent is unhappy with its neighbors, it will move to a new location.

The grid has a certain percentage of empty spaces, which allows agents to move around. The model will also include a slider to adjust the percentage of similar neighbors an agent wants in order to be happy.

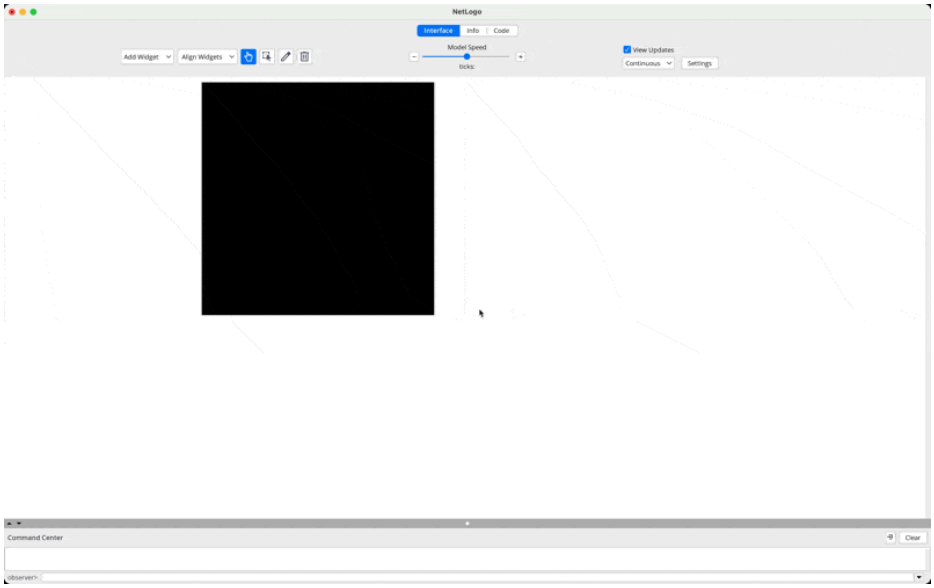
For these reasons, the setup procedure will need to:

1. Clear the environment
2. Define the color of the patches
3. Create a certain number of agents (turtles) with random colors (red or blue) in each patch using the density as probability for an agent to be created
4. Update the characteristics of the agents (color, shape, size) based on their neighborhood
5. Define the global variables and update them based on the initial setup

We will need some global variables to keep track of the model's state. These include:

- `similar-wanted`: The percentage of similar neighbors an agent wants to be happy (controlled by a slider)
- `density`: The percentage of patches that will be occupied by agents (controlled by a slider)
- `happy`: The number of happy agents in the model
- `unhappy`: The number of unhappy agents in the model

Notice that the `similar-wanted` and `density` variables will be controlled by sliders in the NetLogo interface, allowing us to easily adjust these parameters during the simulation.



Also, to account for the happiness of the turtles, we will create a variable called `happy?` for each turtle. This variable will be a boolean (true/false) indicating whether the turtle is happy or not based on its neighbors. We also want to make sure each agent (household) can keep track of the number of similar neighbors, different neighbors, and total neighbors it has. This will help us determine if the agent is happy or not.

Here is the code for the creation of global variables, the breed households, and the creation of attributes for our agents:

```
globals [
    happy-households ; The number of happy agents in the model
    unhappy-households ; The number of unhappy agents in the model
]

breed [households household] ; define a breed for the agents as a
household

households-own [
  happy?          ; boolean indicating if the household is happy
  similar-neighbors ; number of similar neighbors
  different-neighbors ; number of different neighbors
  total-neighbors ; total number of neighbors
]
```

Now we can create the `setup` procedure. This procedure will clear the environment, set the patch colors, create the agents, and initialize their attributes. Also, we want the `setup` to include the initialization of the global variables. Before we start, it is important to note that we will be using procedures (functions) to organize our code better. This will make it easier to read and maintain.

Here is how you can implement it:

```
to setup
    clear-all ; clear the environment
    make-households ; procedure to create the households
    update-households ; procedure to update the shape of the households
based on their happiness
    update-global-variables ; procedure to update the global variables
    reset-ticks ; reset the tick counter
end
```

This is the procedure to create the households:

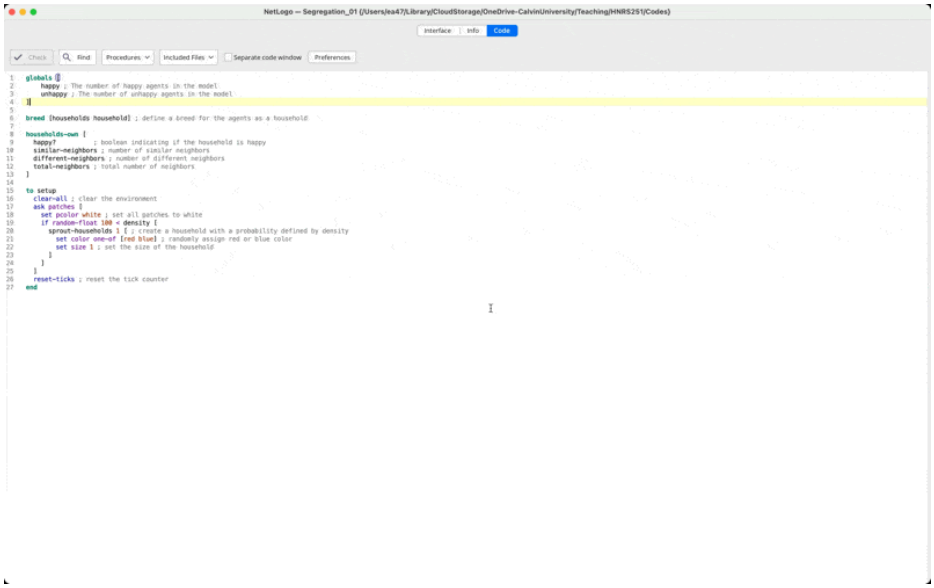
```
to make-households
ask patches [
    set pcolor white ; set all patches to white
    if random-float 100 < density [
      sprout-households 1 [ ; create a household with a probability
defined by density
        set color one-of [red blue] ; randomly assign red or blue color
        set size 1 ; set the size of the household
      ]
    ]
  ]
end
```

For now, we will create empty procedures for `update-households` and `update-global-variables`. We will fill them in the next sections.

```
to update-households
    ; This procedure will be filled in the next section
end

to update-global-variables
    ; This procedure will be filled in a later section
end
```

Finally, we need to create a button in the NetLogo interface to call the `setup` procedure.



Now you can run the `setup` procedure by clicking the button, and it will initialize the model with a grid of agents (households) randomly assigned as red or blue, based on the density slider.

4.m.ii. *2. Update Households:*

We turn now to the aesthetic aspects of the model. Using arrowheads makes it hard to see the state of the agents (if they are happy or not). We can think of many ways to represent the state of the agents. Let's remember that the color represent the households opinions/ideas/ideologies (red or blue). We can use the shape of the agents to represent their happiness. For example, we can use circles for happy agents and squares for unhappy agents. This way, we can easily see the state of the agents in the model.

For this model, we will use an x for the unhappy agents and a square for the happy agents. That is for the sake of visualization. You can use any shape you want. The important thing is to be consistent and to use shapes that are easy to distinguish.

As the agents will be moving around the environment and changing their state, we will need to update their shape accordingly. We can do this by filling the procedure called `update-households` create before. This procedure will be called any time we need to change the shape of the agents.

```
to update-households
  ask households [
    if happy?   [ set shape "square" ]  ; set shape to square if happy
                [ set shape "x" ] ; set shape to x if unhappy
  ]
end
```

As you will soon notice, we have a problem in here. How do we know if an agent is happy or not? We need to define the rules for happiness. In this model, an agent is happy if the percentage of similar neighbors is greater than or equal to the `similar-wanted` threshold. We can calculate the percentage of similar neighbors by dividing the number of similar neighbors by the total number of neighbors. We also need to make sure that we are not dividing by zero. If an agent has no neighbors, we will consider it as happy.

Let's fix our `update-households` procedure to include the calculation of happiness:

```
to update-households
  ask households [
    set similar-neighbors count neighbors with [color = [color] of
myself] ; count similar neighbors
    set different-neighbors count neighbors with [color != [color] of
myself] ; count different neighbors
    set total-neighbors similar-neighbors + different-neighbors ; total
neighbors
    ifelse total-neighbors > 0 [
      set happy? (similar-neighbors / total-neighbors) >= similar-wanted
    ] [
      set happy? true ; if no neighbors, consider happy
    ]
```

```
    ifelse happy?   [ set shape "square" ]  ; set shape to square if
happy
                [ set shape "x" ] ; set shape to x if unhappy
  ]
end
```

So we now have the setup procedure that creates the households and updates their shape based on their happiness.

4.m.iii. *3. Dynamics:*

So far we have been working on the setup of the model and the update of the households. Now we need to define the dynamics of the model. The dynamics will be defined in a procedure called `go`. This procedure will be called every tick of the simulation. In this procedure, we will ask the unhappy agents to move to a new location. We will also update the shape of the agents based on their happiness and update the global variables.

```
to go
  if all households [happy?] [ stop ] ; stop the simulation if all
agents are happy
  move-unhappy-households ; procedure to move unhappy agents
  update-households ; update the shape of the agents based on their
happiness
  update-global-variables ; update the global variables
  tick ; increment the tick counter
end
```

The procedure to move the unhappy agents is as follows:

```
to move-unhappy-households
  ask households with [not happy?] [
    move-to one-of patches with [not any? households-here] ; move to a
random empty patch
  ]
end
```

Finally, we need to fill the `update-global-variables` procedure. This procedure will count the number of happy and unhappy agents and update the global variables accordingly.

```
to update-global-variables
  set happy-households count households with [happy?] ; count happy
agents
  set unhappy-households count households with [not happy?] ; count
unhappy agents
end
```

No we can go back to the interface tab and create a button to call the `go` procedure. Make sure to set the button to "forever" so that it will keep calling the `go` procedure until all agents are happy. You can also create two buttons to call the `go` procedure (once and forever).

IMAGE HERE

And this is the final code for our Segregation Model. You can now run the model and see how the agents move around the environment until they are all happy.

```
globals [
    happy-households ; The number of happy agents in the model
```

```
    unhappy-households ; The number of unhappy agents in the model
]

breed [households household] ; define a breed for the agents as a
household

households-own [
  happy?          ; boolean indicating if the household is happy
  similar-neighbors ; number of similar neighbors
  different-neighbors ; number of different neighbors
  total-neighbors ; total number of neighbors
]

to setup
    clear-all ; clear the environment
    make-households ; procedure to create the households
    update-households ; procedure to update the shape of the households
based on their happiness
    update-global-variables ; procedure to update the global variables
    reset-ticks ; reset the tick counter
end

to make-households
  ask patches [
    set pcolor white ; set all patches to white
    if random-float 100 < density [
      sprout-households 1 [ ; create a household with a probability
defined by density
        set color one-of [red blue] ; randomly assign red or blue color
        set size 1 ; set the size of the household
      ]
    ]
  ]
end

to update-households
  ask households [
    set similar-neighbors count neighbors with [color = [color] of
myself] ; count similar neighbors
    set different-neighbors count neighbors with [color != [color] of
myself] ; count different neighbors
    set total-neighbors similar-neighbors + different-neighbors ; total
neighbors
    ifelse total-neighbors > 0 [
      set happy? (similar-neighbors / total-neighbors) >= similar-wanted
    ] [
      set happy? true ; if no neighbors, consider happy
    ]

    ifelse happy?   [ set shape "square" ]  ; set shape to square if
happy
                [ set shape "x" ] ; set shape to x if unhappy
```

```
  ]
end

to update-global-variables
  set happy-households count households with [happy?] ; count happy
agents
  set unhappy-households count households with [not happy?] ; count
unhappy agents
end

to go
  if all households [happy?] [ stop ] ; stop the simulation if all
agents are happy
  move-unhappy-households ; procedure to move unhappy agents
  update-households ; update the shape of the agents based on their
happiness
  update-global-variables ; update the global variables
  tick ; increment the tick counter
end

to move-unhappy-households
  ask households with [not happy?] [
    move-to one-of patches with [not any? households-here] ; move to a
random empty patch
  ]
end
```

5. Final Project: Modeling Social Norm Formation and Fragility

> **Warning**
>
> This page is under construction. It is just a draft that will receive a lot of changes yet.

5.a. Project Summary

Students will design an **agent-based model of social norms**—how they emerge, spread, and break down in a community.
The project asks: *How do shared norms of cooperation, fairness, or trust develop in societies, and what forces destabilize them?*

This model invites students to blend technical ABM design with social theory concepts like Durkheim's collective conscience, Goffman's micro-interactions, or Elinor Ostrom's work on commons governance.

5.b. Core Elements

5.b.i. *1. The Model:*

- Agents start with varied dispositions (e.g., cooperative, selfish, conditional).
- Agents interact locally (neighbors, small groups) and update their behavior based on observed norms.
- Students can add *shocks* (e.g., sudden resource scarcity, new external influence, an influx of outsiders) to test fragility of norms.

5.b.ii. *2. Social Theory Integration:*

- Each team selects **one social theorist** (e.g., Durkheim, Weber, Ostrom, Foucault) whose framework they use to guide model assumptions.
- Example:
    - If using Ostrom arrow.r rules for collective resource management.
    - If using Durkheim arrow.r moral consensus as glue for cohesion.

5.b.iii. *3. Experimentation:*

Students run simulations with variations:

- Initial trust levels (high vs. low).
- Network structure (tight-knit vs. fragmented).
- Leadership presence (strong authority vs. decentralized).

They analyze *under what conditions norms are stable* or *collapse into disorder.*

5.b.iv. *4. Deliverables:*

- **Report**: with clear links between social theory and modeling choices.
- **Interactive demonstration**: in NetLogo, Python Mesa, or similar.

- **Reflection section**: How might these simplified models inform our understanding of real-world challenges (e.g., polarization, online communities, campus culture)?

5.c. Why Doing This Work?

- **Hands-on**: Requires real modeling work (parameter design, simulation runs).
- **Interdisciplinary**: Merges social theory with computation in a concrete way.
- **Theological/Philosophical Depth**: Students reflect on the "imperfect models" Smedes highlights—what are the limits of modeling moral/social order in a fallen world?
- **Scalable**: Strong students can push into advanced territory (network science, machine learning for calibration), while everyone can complete a meaningful baseline project.

References

Schelling, T. C. (1971b). Dynamic models of segregation†. *The Journal of Mathematical Sociology*, *1*(2), 143–186. https://doi.org/10.1080/0022250x.1971.9989794

Schelling, T. C. (1971a). Dynamic models of segregation. *Journal of Mathematical Sociology*, *1*(2), 143–186.

Schelling, T. C. (1978). *Micromotives and macrobehavior*. WW Norton & Company.

Wilensky, U. (1999,). *NetLogo http://ccl. northwestern. edu/netlogo/. Technical report, Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston.*